Pull-Request Workflow with Agents

Last modified: 2026-08-29 23:12:14 (PDT)

The practices below apply whether you are driving an agent's work or doing the work yourself. They keep parallel sessions from colliding and keep a pull request moving toward a clean, mergeable state.

File an Issue Before Starting

When starting a **new** piece of work, go **issue-first**: before branching, editing, or opening a PR, make sure a tracking issue exists. Search the tracker first; if no open issue covers the task, **file one** (`gh issue create` / `glab issue create`), then proceed. Never jump straight into a PR without a tracking issue behind it.

The issue is the durable record of intent, scope, and "done" criteria — it gives reviewers context, lets the PR auto-close it via `Closes #N`, and keeps the work discoverable even if the PR stalls. Skip only when the task is already tracked by an open issue.

This rule settles *whether* something is tracked, not *where* it goes. An item whose deliverable is a decision rather than a diff belongs on the discussion board instead, per `choose-issue-or-discussion` — so read "file one" here as "file one in the right venue", which for actionable work is the tracker.

When the issue is a **bug report**, include a minimal reproducible example (a reprex — https://reprex.tidyverse.org/) whenever you can. A reprex is what a maintainer needs to confirm and fix the bug, and it's what they'll ask for anyway, so providing it up front saves a round trip. The `reprexes` skill helps reduce the problem to a minimal, self-contained example.

When filing an issue that contains a list of independent subissues, file each subissue as a child issue linked under the parent (GitHub sub-issues feature: `mcp__github__sub_issue_write` in remote sessions, or `gh api` with the sub-issues endpoint in local sessions).

That splitting rule has teeth, and they are worth stating: a PR's `Closes #N` closes the whole issue, including every item in it the PR never addressed. Read as tidiness,

the rule is easy to skip when the second item feels like a footnote. The actual consequence is that GitHub cannot partially close an issue, so the residual items are not deferred and not reopened — they are silently gone, and nothing in the merge, the PR, or the closed issue reports that anything was dropped.

It is worse than an ordinary lost to-do, because a closed issue is *evidence that the work was handled*. A later reader searching the tracker finds it closed and reasonably concludes every item in it was dealt with, so the loss is not merely silent but actively misleading.

So before writing `Closes #N`, re-read #N and confirm the diff covers all of it. When it doesn't, either split the remainder into its own issue first, or reference the parent with `Refs #N`, which links without closing.

- **Do:** split at filing time, or at the latest before the closing PR merges.
- **Do:** use `Refs #N` when a PR advances an issue without completing it.
- **Don't:** let `Closes #N` ride on an issue whose scope is wider than the diff.

(Morrison-Lab/ai-config#847, 2026-07-29: an issue was filed carrying a primary bug and a secondary note, and the PR fixing the first said `Closes #847`. The second item survived only because the maintainer asked about it before the merge, which is not a mechanism; it was split into #852 and shipped as #853, and both PRs merged within the following half hour. The splitting rule directly above already existed and was simply not applied when #847 was filed, which is the argument for stating its consequence rather than only its instruction.)

Deferring a request out of the current change is allowed, and the tracking issue is what allows it

The rule at the top governs work you are about to start. Its mirror governs work you are declining to start now: a request that arrives while a change is already in flight, and that would grow that change past what it set out to do. Such a request may be deferred, on your own judgment, **provided the deferred item is filed as an issue in the same reply**. The permission and the condition are one rule rather than two. An untracked deferral is not a deferral, it is dropping the request in the vocabulary of scope discipline.

The requests this covers come from the user, which is what makes it worth stating. A reviewer's finding already has a Defer disposition, per `ardi`'s ARD step, and a request the user explicitly defers already routes to `defer-issue`. Neither reaches the commonest case, where the user asks for something adjacent mid-review and the standing instinct treats any direct request as automatically in scope for whatever happens to be open. A request can be genuinely wanted and genuinely out of scope for the current PR at once, and saying so is a service rather than a refusal.

It is a grant of latitude and not an instruction to defer. The default is unchanged: do what was asked. What the grant removes is the bind a mid-flight request creates, where

responsiveness and scope discipline pull opposite ways and doing everything asked is the only move that reads as cooperative.

File the issue so it stands alone, by this fragment's own standard. The conversation that produced the request will not survive it, so an issue reading "do the thing we discussed" defers nothing and only moves the loss somewhere harder to notice.

Say which parts you deferred and why, in the same reply. This is the near-miss, and it reads as compliance from the inside: three things were asked, two were done, the reply describes the two, and nothing states that a third existed. A silent partial delivery is indistinguishable from having done the whole thing, so the user finds out what was dropped only by rereading their own request. Name the deferred item, give the reason, and link the issue.

The boundary with technical debt

`dont-incur-technical-debt` says a filed issue records debt rather than paying it, and that a defect you have already diagnosed inside your own diff is yours to fix now. Nothing here softens that, and the two rules read as contradictory until the boundary is drawn.

That fragment supplies the discriminator, so use its question:

> Does the diff I am about to push contain the thing I just diagnosed as wrong?

When it does, the request is not out of scope, it is the scope, and no issue number buys it out. This rule covers work **adjacent to** the diff instead: pre-existing prose the change never authored, a broader sweep the change happens to touch one instance of, a follow-on improvement that would be welcome later.

- **Do:** defer an out-of-scope request on your own judgment, and file the tracking issue in the same reply that declines it.
- **Do:** name each deferred item, its reason, and its issue, so a partial delivery is visible as partial.
- **Do:** ask the technical-debt question first, and fix rather than defer whatever the current diff itself introduced.
- **Don't:** treat a request as in scope merely because the user made it directly.
- **Don't:** defer without filing — an untracked deferral is a dropped request, and it reads as scope discipline while being the opposite.
- **Don't:** read this as a reason to defer; the default is still to do what was asked.

(Directive from the user, 2026-08-09: "cai: it's ok to defer out-of-scope requests from me; just make sure to track them in issues". It came mid-review on `UCD-SERG/serocalculator#654`, a Quarto methodology-vignette formalization, where adjacent requests kept arriving in quick succession — convert propositions to theorems, sweep the chapter for overclaims, reformat multi-equality display equations — several of them touching prose the PR had never authored.)

Claim a PR or Issue Before Working on It

Before starting a work session on a GitHub PR or issue — i.e. before fetching the branch, making edits, or invoking an automated review cycle — post a brief comment on the PR/issue so other people and any automated review bots know not to start a conflicting parallel session.

Use:

```
gh pr comment <N> --body "Working on this --- paws off until I'm done."
gh issue comment <N> --body "Working on this --- paws off until I'm done."
```

Then proceed with the work. After the session ends (PR merged, issue closed, or work otherwise paused), follow up with a closing comment so the PR/issue is unclaimed for the next person.

Skip the claim step if the most recent comment already says you are working on it **and that claim is still live under the expiration rule below**. This applies to any task that will push commits to a PR branch or run iterative review loops. It does **not** apply to read-only inspection (showing a PR, checking status, explaining a diff) — those don't risk a parallel session.

This includes a PR **you opened yourself**: in repos with an active `@claude` agent (`claude.yml`), the agent can push commits to your branch on PR activity — e.g. merging `main` in — and collide with your in-flight push, so claim early to flag the branch as actively worked. (See `memories/claude-bot-workflows.md`, "(**claude?**) CI action", for the collision-recovery steps.)

When starting work from an issue, follow the claim comment with an immediate draft PR — see `pr-on-claim` for the mechanics. An open PR is a stronger "in-flight" signal than a comment alone.

A claim expires 2 hours after the most recent push or comment on the PR/issue — reassert it rather than resuming under a stale one. A claim comment with no expiry binds the thread indefinitely: a crashed or abandoned session leaves its "paws off" standing forever, and a second session has no rule for when the claim stops blocking. So the convention is time-boxed and keyed to observable activity: a claim is **live for 2 hours from the most recent push or comment** on the PR/issue, and **expired** past that.

The rule cuts both ways.

- **As the claimant:** resuming work after more than 2 idle hours — no push and no comment in that window — starts with a fresh claim comment, not with an edit. The skip-if-already-claimed shortcut above covers only a live claim; an expired claim of your own no longer covers you, because a parallel session is entitled to treat it as lapsed.

- **As a would-be second session:** another claimant's claim whose PR/issue shows no push or comment in over 2 hours no longer blocks you. Take over by posting your own claim comment, never by starting silently — the fresh claim is what flips the thread's state, and it is what tells the stale claimant they were superseded if they return. A claim's age is evidence about the *claim*, not proof the branch is quiet, so the mid-task checks below — the "already done" cross-check against the PR's actual commit list, and the rejected-push tree comparison — still apply before your first push, as does the branch-head re-fetch in the `claim-pr` skill's Notes.

Staleness is decidable by one read rather than by judgment, per `algorithmatize-checks`:

```
gh pr view <N> --json updatedAt --jq .updatedAt        # VIEW_PR
gh issue view <N> --json updatedAt --jq .updatedAt     # VIEW_ISSUE
```

`updatedAt` moves on more events than pushes and comments (labels, reviews, body edits), so it only ever **over-approximates** freshness: a stale verdict from it is definitive, and a borderline-fresh one defaults to respecting the claim — the safe direction, since over-respecting a dead claim costs a wait while under-respecting a live one costs a collision.

- **Do:** post a fresh claim before resuming work when more than 2 hours have passed since the most recent push or comment on the PR/issue.
- **Do:** treat another session's claim as expired on the same 2-hour reading, and post your own claim before touching anything.
- **Don't:** read "the most recent comment already says I'm working on it" as a standing skip — that shortcut covers only a claim under 2 hours old.
- **Don't:** start work under an expired claim, your own or anyone else's, without a fresh claim comment — a silent resumption and a silent takeover collide identically.

(Directive from the user, 2026-08-15: "let's set a convention that pr and issue claims last 2 hours from the most recent push or comment; if it's been longer than that, reassert your claim.")

Verify a mid-task "already done" claim against real PR state before trusting or redoing it. A PR you claimed and are actively driving can still gain commits from a **second, independently-running session** under the same account — a `<github-webhook-activity>` review-comment-reply event can describe work ("Addressed… Pushed in `<sha>`") that this session never did. Don't assume it's fabricated or injected, and don't reflexively redo the same fix: cross-check the PR's actual commit list (`gh pr view --json commits` / `pull_request_read get_commits`) and review threads before either (a) trusting the claim, or (b) starting the same fix yourself. If a commit with that SHA genuinely exists, authored close to when the event arrived, treat it as confirmation a live parallel session owns this PR right now — stop pushing further speculative fixes yourself, and, if genuinely in doubt, ask whether to keep driving or step back, rather than racing the other session's pushes. This gap is distinct from the initial claim check above: it's not about claiming a PR before starting, but about **re-verifying you're still the sole active driver** once work has been under way for

a while — especially when you picked up the PR mid-session (e.g. by answering a diagnostic question about it) rather than through the normal claim-then-branch flow, so no fresh "paws off" check ever ran right before you started pushing. (`d-morrison/gha#286`, 2026-07-24: a webhook event delivered a review-comment reply attributed to `d-morrison` reading exactly like a Claude-authored reply, claiming a fix "Addressed… Pushed in 3fb8c5b" that this session hadn't made; verified real via `get_commits` before proceeding — a second live session, not injection.)

The git-level variant of that check: a rejected push whose remote commit is byte-for-byte what you were about to push. The section above covers a *comment* claiming work was done. Here the parallel session makes no claim at all. Your `git push` is simply rejected because it pushed first, and what it pushed is the same merge you just made. The reflex on a rejected push is to merge again, which would stack a redundant merge commit on top of an identical one.

Four reads settle it before you touch anything:

```
git rev-parse HEAD^{tree}                  # your merge's tree
git rev-parse origin/<branch>^{tree}       # theirs
git show -s --format=%P HEAD               # your merge's parents
git show -s --format=%P origin/<branch>    # its parents
```

An identical tree plus identical parents means the two merges are the same merge, so the right action is `git reset --hard origin/<branch>`.

- **Do:** compare trees and parents before deciding what a rejected push means.
- **Do:** discard your local merge with `git reset --hard origin/<branch>` once both match.
- **Don't:** re-merge reflexively on a rejected push — that is what produces the redundant merge commit.
- **Don't:** force-push over the other session's commit.

(`Morrison-Lab/ai-config#965`, 2026-07-31: `main` moved one commit, a local `git merge origin/main` was made, and the push was rejected. The remote carried `b8d2273`, a merge of the same two parents, with tree `1bda1bc`, identical to the local merge's.)

Matching parents with a differing tree means the two sessions resolved the same merge differently — merge the two commits together, don't reset onto either one. Identical parents but a different tree is not the "same merge" case above: a concurrent session (a human, or an `@claude`-style bot reacting to PR activity) merged the same `main` commit into the same branch, but resolved a real conflict (or made an additional fix) differently than you did. `git reset --hard origin/<branch>` here silently discards whatever your version got right that theirs didn't — e.g. a version-parity bump their merge didn't carry, or a merge-conflict resolution theirs got wrong. Instead, fetch and merge the remote branch into your local one

(an ordinary three-way merge, since the two commits share both parents as a common ancestor between them); resolve any conflict on its own merits, the same as any other merge, then push the result.

- **Do:** compare parents first, then trees; matching parents with a differing tree calls for a merge of the two commits, not a reset.
- **Do:** merge the remote branch in normally, resolving whatever differs on its own merits.
- **Don't:** `git reset --hard` onto a same-parents-different-tree remote commit — that discards your own resolution outright, on the assumption the two were interchangeable.

(`UCD-SERG/serocalculator#654`, 2026-08-08: `main` had absorbed a same-version dev bump from an unrelated PR, so this session merged `main` in and bumped the version past it while separately resolving a real conflict in `inst/WORDLIST`. The `@claude` review bot's own `main`-sync had pushed a merge with the same two parents in the meantime, but it left the version at parity — still failing `version-check` — and had never seen the `WORDLIST` conflict at all, since its sync predated that conflict existing. Merging the two commits, rather than resetting onto either, kept both fixes.)

Handing off mid-task to another agent, on user request ("finish what you're doing, then relinquish holds; I'll put another agent on them"): don't just stop — leave the next agent a clean starting point. On each claimed PR/issue: (1) post a status comment on the PR itself distinguishing what's **done** from what's genuinely **not done** (the actual point of the issue, not just the side-fixes found along the way) and any blocker still open, so the next agent doesn't have to re-derive it from the diff; (2) post the closing/unclaim comment on the issue per the pattern above; (3) `unsubscribe_pr_activity` (or stop babysitting locally) so you don't keep auto-fixing a PR you no longer own; (4) stop any background watch/poll task tied to that work (e.g. a `ScheduleWakeup` or a `Monitor`/background-Bash wait) so it doesn't fire into a session that's moved on. A merge-conflict-free `git status` and a pushed branch are not enough on their own — the status comment is what makes the handoff legible. (ucdavis/bcs `gia` session, 2026-07-06: handed off PRs #310 and #311 mid-implementation this way, each blocked on the same slow `renv::restore()`.)

Keep Your Branch Synced with Main

Whenever `main` has moved ahead of a PR branch you're working on, **merge `main` into the PR branch** before the next push or review trigger. Don't wait for a conflict to surface or for someone to ask.

Worked-example case records for the rules below live in sync-with-main.cases.md, moved out of the auto-loaded context.

This fragment covers the single-branch-vs-`main` case. When orchestrating a multi-agent `ultracode` session, merges can happen at more points than that — see `ultracode-merge-conflicts` for the broader check (worktree-isolated agent branches, concurrent `parallel()` results) and the note on GitHub's mergeable indicator not evaluating custom `.gitattributes` merge drivers.

Always check for merge conflicts with main before pushing results to remote. Run this before every push, not just before triggering a review:

```
git fetch origin main
git log --oneline ..origin/main | head    # any commits? main is ahead --- merge it in
git merge origin/main
```

If the push is rejected because `main` has moved (`! [rejected]` with `(fetch first)` or `(non-fast-forward)`), fetch and merge before retrying — don't force-push.

Always do this before triggering a fresh review too, so the reviewer evaluates the PR against current `main` rather than a stale snapshot.

Don't rebase or squash-rewrite a published PR branch unless explicitly asked — a merge commit is the right move because it matches GitHub's "Update branch" button and preserves the PR history.

If the merge has conflicts, resolve them, run the project's standard pre-commit checks (render / lint / spell / tests), commit, then push. Don't push a half-resolved merge.

After merging main, re-check version parity. In R packages with a `version-check` CI job, the branch's `DESCRIPTION Version:` must *exceed* main's. A conflict-free merge can silently put them at parity — main advanced (e.g. another PR merged between when you last bumped and now). After every merge of main, compare versions:

```
git fetch origin main
git show origin/main:DESCRIPTION | grep ^Version
grep ^Version DESCRIPTION
```

If they match, bump the branch's `Version:` by one patch level before pushing.

Re-check `main` again right before the final push, not just at the start of a merge. Resolving a conflict (rerunning generators, fixing prose, updating a CHANGELOG entry) can take long enough for `main` to advance a second time. A `git fetch origin main` immediately before `git push` — after conflict resolution is done, not only before it started — catches that case; an earlier CI failure on a commit you thought was current is a symptom of skipping this second check.

A conflict-free merge does not mean derived artifacts are in sync. If your branch regenerates a generated tree (e.g. `codex-skills/`, a lockfile, rendered docs) and `main` added a

new *source* input the generator consumes (a new skill, a new dependency), git merges both cleanly — but the generator never ran against the new input on your branch, so its output is missing or stale and the sync check fails on `main` after both land. After merging `main`, re-run the generator and commit the result whenever main touched the generator's inputs — don't trust the absence of conflicts. (Concretely: merge the PR that adds the new skill *first*, then sync the wrapper-regenerating branch and rerun `scripts/sync-codex-skill-wrappers.py` before merging it.)

A CI failure on a brand-new PR's very first commit (e.g. the empty claim-commit from `pr-on-claim`) is a signal to check `main`'s position before debugging the failure itself. A local checkout that sat around since before the session started can already be many commits behind `main` — the failure (a stale generated-tree check, a check `main` has since added or dropped) often isn't a real problem with your change at all, just `main` having moved. `git fetch origin main && git log --oneline ..origin/main` first; if `main` is ahead, merge it in and re-run the checks before treating the failure as something to fix in the diff.

The same staleness trap has a silent variant with no CI failure to flag it: a worktree/branch named after a PR's followup can still be based on a `main` from before that PR actually merged. A worktree directory or branch name suggesting "after PR #N" (e.g. `pr-N-followup-...`) is not proof the branch's actual base commit postdates #N's merge — it can have been created earlier and simply named for its intended purpose. Trusting that naming, then reasoning from `git show <hash>` for a commit found via `git log --all` (which lists every reachable commit across all refs, not just your branch's ancestry) can make content look present when it isn't actually in your branch yet. Verify with `git log --oneline HEAD..origin/main` or by reading the actual blob your branch would produce (`git show HEAD:<path>`, or the working tree itself before assuming what it contains), not a commit hash pulled from `--all`. If `main` has moved, merge it in before building further edits on the assumption the missing content exists.

A real conflict inside a file whose logic is also copied elsewhere (an extracted script, a doc example) needs the copy re-synced too, not just the conflicted file resolved. When a PR extracts inline logic (e.g. a workflow step's shell block) into a standalone script for testability, and `main` independently changes that same inline logic while the PR is open, resolving the merge conflict in the workflow file is not enough — the extracted script must be updated to match `main`'s new logic exactly, or the PR silently reverts `main`'s fix the moment it merges. Diff the extracted copy against `main`'s current inline version line-for-line (strip indentation, `diff`) to confirm an exact match, not just "looks about right." If the PR carries tests against the extracted copy (fixtures, unit tests), add regression coverage for whatever `main`'s change fixed — the merge is the natural moment to catch a gap the original PR's tests didn't anticipate, and to prove the new fixtures actually catch the regression (temporarily revert the fix, confirm the test fails, then restore).

When `main` DELETES a file your branch references, resolving the marked conflict is not enough — grep the whole tree for the deleted path. Git only conflicts where both sides edited the same lines, so a merge that brings in a deletion flags the file that *used*

the thing, and nothing else. Any other reference to the deleted path — a docstring citing it as precedent, a comment, a doc cross-reference — merges cleanly and silently becomes a dangling reference, because those files were never in the conflict's scope. After resolving any merge that removed a file, run `grep -rn "<deleted-path>"` across the repo and re-point or reword each hit; then distinguish live references (must be fixed) from historical citations of the removal itself (correct as-is, leave them). This is the deletion counterpart to the extracted-copy case above: there the logic moved and a copy went stale, here it vanished and the pointers went dead.

A textual conflict in a skill file can be the symptom of a conceptual duplicate, not just competing edits to the same line. When merging `main` into a branch that's authoring a new skill, if the conflict lands in a `## Relationship to other skills` section (or `main` added an entirely new skill in the same territory), that's a signal to re-run `skill-builder`'s Step 0 judgment — not just resolve the diff mechanically. Compare the new skill against whatever landed on `main`: are they the same concern (fold into one, redirect), or genuinely distinct (cross-link both directions so neither reads as an unexplained near-duplicate)? `skill-builder`'s in-flight-work scan only runs once, at the start; `main` can grow a colliding skill in the time a PR is open, so the check has to be repeated at merge time too.

The same collision can land before you write a line, and then it produces no conflict at all — just duplicated work nobody flags. The bullet above catches a duplicate at merge time, via a conflict. When `main` gains the colliding content while your change is still *planned* rather than written, there is nothing to conflict with: you write the duplicate, push it, and the review has to argue you out of content that was already redundant on arrival. So re-run the dupe check after any fetch that brings in new commits, not only at merge — a plan researched an hour ago was researched against a different `main`.

The cheap version is to read what actually arrived rather than only the count: `git log --oneline <old>..origin/main` plus `git diff --stat` over the same range, then ask whether any of it covers something still on your list. In a session that loads skills or plugins from the repo, a new one appearing in the session's own skill listing is the same signal arriving for free.

This is a *timing* gap, and it composes with the *scope* gap rather than replacing it. `check-open-prs-before-duplicating` covers work that is still in flight, unmerged, and therefore invisible to any check against `main`; run that one too, since a duplicate is just as wasted whether the collision has landed yet or not. Both checks share the same weakness — each runs once, at the start, and answers for the moment it ran.

Dropping the planned work is the cheap outcome, so record why in the issue and the PR body rather than deleting it silently — otherwise the next person re-proposes it.

A routine merge from `main` can create the duplicate inside your own diff. The collision above lands before you write, so the duplicate is redundant on arrival. A later `main` merge is quieter: both branches were non-duplicative when they were written, and the duplicate appears only when you bring the other branch's text into yours. Git reports a clean merge because the two copies sit in different files. Diff-scoped added-line checks do not help either, because the duplicated lines already existed on one side or the other. So after merging `main`

into a prose branch, run the duplicate check against the branch's full current diff and the neighbouring corpus, not only against lines added by the merge commit.

- **Do:** after a `main` merge, re-run a cross-file duplication check over the merged branch's whole prose diff.
- **Do:** treat a reviewer finding on such duplication as correct even when each copy was independently right before the merge.
- **Don't:** assume a conflict-free `main` merge preserved DRY, or that the duplicate would have appeared in an added-lines-only scan.
- **Don't:** answer by asking which branch "introduced" the duplication; the merge introduced the state that made both copies coexist.

Two PRs that each append a new terminal numbered subsection to the same file (e.g. `### 5. ...` in a `CLAUDE.md` review-guidelines list) will conflict on merge even when neither side's content actually disagrees. This isn't an editorial clash — it's two authors both writing to "the next number" at the same insertion point. Resolve by keeping **both** additions and renumbering sequentially from the collision point on, not by dropping either side; then grep the file for any other place that names the old numbering (a cross-reference, an index). This is also a reason `fully-clean`'s CI-green-and-review-clean verdict is a snapshot, not a mergeability guarantee — `main` can pick up its own append in the same spot after your last review round, so a PR can go from "reviewed clean" to "needs a merge conflict resolved" with no defect in its own diff. Before reporting a PR ready to merge, re-check with `git fetch origin main` plus the `git merge-tree` command from `resolve-conflicts`, not just a cached `mergeable` flag or an earlier green CI run.

After merging a PR that extracts an inline block into a reusable unit (a composite action, a shared script/function), check other open PRs that still edit that same inline block — your merge just broke their textual diff, even though their intended change is usually trivial to re-apply to the new location. This is the mirror image of the case above: there, you're the one resyncing after `main` moved a copy of your logic; here, *you* are the one who moved the logic, so the burden of noticing and fixing the resulting conflict falls on you, not on the sibling PR's author waiting to hit it. Don't wait for that PR's own merge/CI to surface the conflict — check every open PR touching the same file right after your extraction merges: `git merge-tree "$(git merge-base origin/main origin/<sibling-branch>)" origin/main origin/<sibling-branch>` (or `gh pr diff <N>` against the new `main`) shows whether it still applies cleanly. Re-apply the sibling PR's actual semantic change (not a mechanical `--theirs`) to the new location, verify with a direct diff that the extracted unit now differs from `main` by exactly that PR's intended change and nothing else, then push to their branch and flag what you did in a PR comment.

See `sync-with-main.cases.md`, "Check other open PRs after merging an extraction".

That "push to their branch" is scoped by standing, not only by cause. gha#201/#202 were CI workflow files in a repo the author drove, where a push saves the sibling's author a round and risks nothing they were relying on. The same push onto a branch you do not own —

a colleague's active work, and most sharply a release branch carrying an out-of-band process — can disrupt something a comment would not. There, name the extraction, the deletion, or the rename and where the content went in a PR comment, and leave the push to whoever owns the branch. Causing the conflict obliges you to *surface* it. It does not by itself license editing someone else's branch. See `batch-merge-and-resolve`, "A conflict your sweep found is not a conflict your merge caused", for the attribution step that says which conflicts are yours in the first place.

An add/add conflict on a *shared config file* usually means two PRs independently fixed the same root cause — reconcile the reasoning, don't just pick a side. This generalizes the skill-file case above beyond skills: a repo-wide CI/lint/build config fix (a new tool config file, a workflow tweak) is exactly the kind of change multiple sessions or bots are likely to attempt in parallel once a check starts failing on `main` for everyone. When the conflict is a whole-file add/add (not just competing edits to an existing file), read both sides' reasoning — code comments, commit messages, the PR discussion — before resolving; usually one side's explanation is more complete (covers a case the other missed, cites the tool's actual constraint) and should win outright rather than mechanically merging fragments of both. Re-diff the PR against `origin/main` after resolving to confirm the PR's remaining changes are its own original scope, not a reintroduction of what the other, now-merged PR already added.

A `dirty` `mergeable_state` on a bot-opened PR can mean a sibling PR already closed the same issue, not just that `main` drifted. An issue-triggered `@claude` workflow can fire twice on the same issue in quick succession (a duplicate dispatch, or two people independently routing the same request), producing two independent PRs that both fully resolve it — including adding the identical new file. The second PR's merge conflict is an add/add on that new file, and it looks like ordinary main-drift, but treating it that way and mechanically resolving in favor of "ours" silently reintroduces a duplicate the other PR's merge already published. Before resolving, check the PR's linked issue for **other** cross-referenced PRs/closing events — if one already merged and closed it, diff the conflicting file against `main`: if it's the sibling PR's already-published version, keep `main`'s content and keep only this PR's genuinely distinct remainder (a piece the sibling PR never did), rather than re-adding a second copy.

The same parallel resolution can be a whole-file split, and then files can vanish from your diff with no deletion hunk to read. The add/add and duplicate-issue cases above both say to keep `main` when a sibling PR already published the same new file. The split case adds a second check, because resolving the one conflict can also make other files disappear from your PR's diff entirely. Those files look harmlessly gone, and there is no deleted line for `ardi`'s pre-push deletion sweep to inspect. Two causes are indistinguishable from the final diff alone: `main` absorbed your cross-reference edit, or the merge dropped your work. So verify each vanished file against the pre-merge head before calling the collapse correct. For each file that left the diff, compare the original head against the merge-base to recover what your branch intended, then confirm current `main` now carries that same change. Search `main`'s whole corpus for that change rather than the path it used to live at: a sibling PR that relocated the content — a companion-file split, a rename, a section moved between files — leaves it present

but elsewhere, so a path-scoped confirmation reports it missing and invites you to re-add a copy `main` already has. ardia's `Superseded` terminal state carries the measurement. Only after that per-file check is it safe to treat the smaller diff as a successful conflict resolution rather than as lost work.

- **Do:** save or read the original pre-merge head, list the files that left the PR diff after the merge, and verify each one's intended change is already on `main`.
- **Do:** keep `main`'s version for the overlapping split file when the sibling PR has already published the same refactor, then carry forward only this PR's distinct remainder.
- **Don't:** infer that a vanished file was safely absorbed merely because the final diff got smaller.
- **Don't:** rely on the deleted-lines sweep for this case; content that left the diff has no deletion hunk for that sweep to show.

When the whole PR is superseded, not just one file, the conflict is telling you to close it rather than resolve it. The two cases above keep `main`'s version of a file a sibling PR already published, and carry forward the current PR's distinct remainder. The remainder can be empty. When a `main`-merge conflict pits *every* added line of an idle PR against a better-formatted copy already on `main` — a sibling PR having landed the same content — resolving toward `main` leaves nothing, and the PR's own prior review findings are moot. Confirm by grepping `origin/main` for the PR's distinctive added phrases before resolving anything: all present means superseded. The right action is then to recommend closing the PR, since its content is preserved on `main`, not to push an empty diff to a clean verdict. For an ARDIA sweep this is a terminal state of its own — see ardia's `Superseded`, which also gives the up-front check that catches it before rounds are spent.

A merge into a growing numbered list (e.g. gha's `CLAUDE.md` "Code review guidelines" section) can produce zero blank lines between two adjacent headings even with no textual conflict — lint catches it, git doesn't. When a section is a hotspot several PRs independently append items to (each PR adding its own `### N.` block at the end), a clean three-way merge can still splice one PR's closing line directly against the next PR's heading with no blank line between them — this doesn't produce a `<<<<<<<` conflict marker (git resolves it as a straightforward insertion), so it's easy to push without noticing. `markdownlint`'s MD022 (blanks-around-headings) is what actually catches it, as a CI failure with no proximate code change to explain it. Re-run the repo's markdown lint (or at minimum re-read the diff around every `### N.` boundary you didn't personally write) after any merge that touches a shared growing list, not just after a merge with conflicts.

The same splice happens to LIST ITEMS, and there `markdownlint` most likely does NOT catch it — so nothing turns red at all. The case above is a heading spliced against preceding text, which MD022 decides. The changelog case is a *bullet* spliced onto the previous item's continuation line:

```
  `data-raw/precompute-true-effects-chunk.R` (#429).
* The `docs` workflow's "Build site" step no longer times out intermittently.
```

That is a valid **tight** list item, so a list in which every other entry is blank-line separated silently starts mixing tight and loose items and renders inconsistently. `markdownlint`'s blanks-around-lists rule governs the boundaries *of* a list, not the gaps *between* its items, and no default rule enforces consistent looseness within one list — so unlike the heading case, CI stays green and only a human reading the merged section notices.

Check it mechanically instead of by eye; one line decides it:

```
awk 'prev !~ /^[[:space:]]*$/ && /^[*+\-] / {print FILENAME":"NR": "$0} {prev=$0}' NEWS.md
```

Use `[[:space:]]*` rather than a bare `/^$/` — a whitespace-only preceding line is not a violation and produces false positives. The pattern `[*+\-]` covers all three common Markdown unordered-list markers; `^\*` alone would miss `-` and `+` bullets.

Two consequences. Run this after any merge into a changelog or other growing bulleted list, alongside the heading check above. And note that a `merge=union` driver on such a file (see configure-gitattributes) *increases* the rate of this defect, since union resolves an append collision by keeping both sides with no conflict to review — so confirm a detector is wired into CI before enabling one, not after.

Run that check as a whole-file count, and compare it before and after — scoping it to the lines you added cannot see this defect at all. The check above is the right instrument; the natural way to apply it is the wrong one. Having found the file's spliced bullets, the obvious next question is which of them are yours, and the obvious way to answer is to intersect them with the lines the branch added. That question is unanswerable, because the defect is a **deleted blank line** before a bullet that was already there. The bullet is *context* in the diff, never an addition, so the intersection is empty by construction and the check reports a confident zero.

Note how this differs from the scope failures elsewhere in this corpus, where a check's **inputs** were too narrow — a glob, a missing flag, a two-dot range. Here the inputs were right and the **question** was wrong, which no widening fixes. And it fails in the direction that reads as an all-clear, on the one file a reviewer will not re-derive.

The sound form is a count delta over the whole file, which needs no judgment about ownership and no diff at all:

```
git show origin/main:NEWS.md   | awk '...' | wc -l   # before
awk '...' NEWS.md              | wc -l              # after
```

A merge must not increase the count. That is an `algorithmatize-checks` instrument in the strict sense — two integers decide it — and it holds whoever authored the surrounding lines.

Generalize past changelogs, because the property is about the defect rather than the file: **when a defect can be introduced by deleting a line, any instrument keyed on added lines is unsound.** Ask instead whether a whole-file measurement got worse. The version-parity rule above is the same shape — a conflict-free merge leaves the branch at parity with `main`, `version-check` goes red, and there is nothing in the diff to point at — which is why that rule is a direct comparison of two `DESCRIPTION` versions rather than a diff inspection.

The shared trigger is the practical part: **a conflict-free merge is exactly when nothing prompts anyone to look.** Both defects arrive through one, both are invisible to diff-scoped checking, and the merge reports success.

- **Do:** measure the whole file before and after a merge, and treat any increase as the merge's fault regardless of who wrote the lines.
- **Do:** ask, of every diff-scoped check, whether the defect it targets could be caused by a deletion — and replace it with a count if so.
- **Don't:** intersect a whole-file finding with the branch's added lines to decide ownership; for a deletion-caused defect that always returns zero.
- **Don't:** read a conflict-free merge as a merge that changed nothing beyond what the diff shows.

A commit claiming "I've pulled main and resolved the merge conflicts" can be lying — verify it actually merged before trusting the claim. A genuine conflict-resolution commit is a merge commit (two parents); a commit that just hand-edits files to *look* resolved, without running a real `git merge`, is an ordinary single-parent commit — and it never actually incorporates whatever new state of `main` prompted the "resolve conflicts" request in the first place. This is easy to miss because GitHub's own `mergeable`/`mergeStateStatus` fields don't distinguish the two: both look identical from the PR page until you check the commit graph. Verify with `git show -s --format="%P" <commit>` — one hash means no real merge happened, regardless of what the commit message says. If a branch needed conflict resolution more than once and each attempt claimed success but the PR still shows `CONFLICTING`, check every "resolved conflicts" commit in its history this way before trying yet another resolution attempt on top of a foundation that was never actually re-merged.

Driving a Pull Request to Clean

Whenever you are working a PR/MR, run the full **ARDI** loop by default, without being asked: **A**ddress every flagged item, **R**ebut findings that are wrong, **D**efer out-of-scope items to tracked issues, then **I**terate with a fresh review — repeating until the latest review is **fully clean**. Don't stop at "review-clean, just needs approval" and hand triage back; keep the cycle going until it's genuinely clean.

Extended rationale — the mechanism, evidence, and argument behind each rule below — lives in `ardi.rationale.md`, moved out of the auto-loaded context. Each rule here keeps its statement and its Do/Don't pair; read the companion when the reasoning or the evidence is the question.

Worked-example case records for the rules below live in `ardi.cases.md`, moved out of the auto-loaded context.

Continuously monitor every PR/MR you are actively working until it reaches that terminal state.

That wait is conditional on a run having been scheduled, and on some repos a push schedules nothing.

- **Do:** read the review workflow's `on:` block before the first push to a PR in an unfamiliar repo, and dispatch explicitly after the round's last push when it carries no push-based trigger.
- **Do:** treat a non-zero `check-pr-fully-clean.py` on a dispatch-only repo as a prompt to dispatch.
- **Don't:** read green CI at the current head as evidence a review is in flight — on a dispatch-only repo that is the steady state, not a transient one.
- **Don't:** let a verdict from an earlier head stand because the repo's trigger class was already known. Knowing it is not the same as acting on it each round.

Dispatching needs no permission, so do not ask about the spend.

The rule above is about the mechanism and says nothing about authorization, which leaves a session free to know it and still stall — by reasoning that a review round costs money and the spend is the maintainer's call. That sounds like restraint and is indistinguishable from it from the inside.

The asymmetry runs the other way. A green, unreviewed PR is **parked, not clean**, so declining to dispatch does not save a round; it holds the PR in a state that reads as finished and is not. The stall also spends the user's attention every time, which is the thing the review loop exists to conserve.

So dispatch when the round is ready, and put the run in the status report rather than the question.

- **Do:** dispatch the review yourself once the round's last push has landed, on any repo whose reviewer is dispatch-only.
- **Do:** name the run you are waiting on, so the report carries a fact rather than a request.
- **Don't:** write "spending a round is the maintainer's call" into a status report, or hold a ready PR pending a spend question.
- **Don't:** read this as a general spending grant — it covers scheduling a review, and merging is still `mwc`'s to govern.

(Directive from the user, 2026-08-16: "always dispatch". Three PRs reached green CI on a `workflow_dispatch`-only repo in one session, and each time the session asked before dispatching, citing rounds that had billed $12.14, $10.37 and $12.44 against a monthly limit already reached. Both earlier dispatches came back `Needs more work`, one with a blocking correctness bug, so the round was not a formality. Tracked as ai-config#1571.)

Dispatch once, after the round's LAST push — a per-push rhythm cancels its own reviews.

Dispatch with `--ref <PR-branch>`, or the resulting failure is invisible on the PR.

- **Do:** finish pushing, then dispatch once, and name in the status report which run you are waiting on.
- **Do:** pass `--ref <PR-branch>` on every dispatch, so the review's check runs attach to the PR head.
- **Do:** diagnose a missing verdict by reading the run, since a cancelled dispatched run leaves no trace on the PR's check-run list.
- **Don't:** dispatch per push — each one cancels the last, and the round spends review time producing nothing.
- **Don't:** re-dispatch reflexively when a verdict is missing. If one is in flight, the retry cancels it.
- **Don't:** read a green, nothing-pending PR as reviewed on such a repo — that is also what an invisible cancelled gate looks like.

A cancelled run is invisible to a session READING the PR and loud to one SUBSCRIBED to it, and the second is the dangerous direction.

The bullet above is scoped to the check-run list deliberately. GitHub lists check runs for the **head commit**, so a run cancelled after the head moved leaves a `require-review` failure hanging off the superseded SHA, where a session reading the PR will not find it.

The webhook stream is a different surface and it does not filter that way. The cancel fires a `check_run.completed` with `conclusion: failure`, so a session subscribed to PR activity is woken by a red **required** check on its own PR.

That inverts the risk the bullet above describes. An invisible failure costs you a verdict you thought you had, and you find out by looking. A visible failure on a superseded commit costs more, because the drive-to-green posture says not to end a CI-failure wake without pushing a fix or replying with a blocker — so the reflex is to fix, against a commit that is no longer in the PR's timeline. At best that is wasted work. At worst you change the current head on the authority of a red check that was never about it.

One field decides it, and it is the field `fully-clean` already names for the neighbouring problem: compare the event's own `head_sha` against the PR's current head. Equal means act. Unequal means confirm a run is live at the real head, and leave the diff alone.

- **Do:** compare a CI-failure event's `head_sha` against the PR's current head before diagnosing anything.
- **Do:** reply naming the superseded SHA rather than staying silent, so the wake is visibly dispositioned rather than dropped.
- **Don't:** read "leaves no trace on the PR" as covering the webhook stream — it describes the check-run list, which is filtered by head commit.
- **Don't:** push a fix in response to a red check whose `head_sha` is not the head; the check is not about the code you would be changing.

See `ardi.cases.md`, "A cancelled dispatch that fired a failure webhook against the superseded SHA".

See `ardi.cases.md`, "A per-push dispatch cancels its own review, invisibly".

A dispatch you make while idle can still be cancelled — by a push you did not make. The per-push rule above governs your own pushing rhythm colliding with your own dispatch. The concurrency group is keyed on the PR number alone, not on who triggered the run or how, so a **third party's** push into the same PR branch cancels your dispatch exactly as a push of your own would — even when you have pushed nothing since dispatching. On a repo with an active `@claude` agent, the bot itself is such a third party: reacting to review activity, it can push a `main`-sync merge into the PR branch (see `claim-pr`'s note on this), which is a `synchronize` event and, on a repo that reviews on push, schedules a fresh review run in the same group — cancelling yours.

The tell: your dispatched run reads `cancelled`, its `headBranch` is whatever ref you dispatched against — the default branch if you omitted `--ref`, the PR branch if you passed it, so this conjunct alone proves nothing — and a newer `pull_request`-event run exists for the same PR, at a newer head. The right response is to do nothing — the cancellation is benign, since the newer run supersedes at a better head, and re-dispatching would cancel *that* one instead.

- **Do:** before treating a cancelled dispatch as a lost review, check for a newer `pull_request`-triggered run on the same PR. Its existence and its head are the whole explanation.
- **Do:** re-fetch the PR branch head before concluding a dispatch failed — a head that moved without you pushing explains a cancellation the per-push rule above cannot.
- **Don't:** re-dispatch to "fix" a cancelled run that a newer, better-placed run already superseded — that cancels the survivor instead.
- **Don't:** read `require-review: failure` on the cancelled run as a live problem. It is a side effect of the superseded run, and the newer run supplies the real verdict.

See `ardi.cases.md`, "A third-party push cancels an idle-dispatched review".

Always execute `python3 scripts/check-pr-fully-clean.py <pr>` synchronously in the foreground turn to evaluate clean verdicts. Whenever ending a turn while waiting for an AI review or CI completion on an active PR after pushing code, launch a `schedule` timer (e.g. 120s) to check back. When the timer fires: - Check if a review for the HEAD SHA has arrived. - If no

review has posted yet, verify whether review workflow runs are still in progress (`gh run list` / `gh pr view --json statusCheckRollup`). - If review workflows are still running: schedule another timer to check back. - If the reviewer failed, was canceled, skipped with no replacement (e.g. quota limit), or produced a stub review with no stated verdict: invoke self-review fallback per `self-review-fallback.md` rather than stalling the loop. - Otherwise, fix any underlying workflow or dispatch issues discovered along the way and schedule another timer to maintain continuous monitoring until a review lands, self-review fallback triggers, or CI completes. This applies transitively to PR-driving workflows such as `gi`, `gii`, and `ardia`; only monitor PRs the session owns or has explicitly claimed, so the rule does not authorize changing someone else's work.

The loop's terminal action is to **report the PR ready, not to merge it**. Merging is human-gated — it happens only on an explicit human "merge it" (the `merge-it` skill), never as a step ARDI takes on its own. So when you carry a PR across a `ScheduleWakeup` or `/loop` wait, **never** bake a self-merge directive like "if clean and CI green, merge it" into the wakeup/loop prompt: a scheduled prompt fires back as a user-role turn, so a self-authored "merge it" only *looks* like human approval (and Claude Code's auto-mode classifier will rightly deny it as a self-authored merge). Drive to fully clean, report ready, and leave the merge — and any other destructive one-off, e.g. a `gh workflow run` that force-pushes — for explicit human authorization.

Because the loop ends there, **the clean verdict is also where `ums` runs** — don't hold the pass for the merge, which is on the human's clock rather than this session's and may land after a `/clear` or not at all. See `CLAUDE.md`'s "Run UMS proactively, as learnings accumulate"; the merge-time pass in `post-merge` then only has to cover what the merge itself taught.

The one exception: if the human has explicitly granted the `mwc` (merge-when-confident) session permission, that grant is a live human instruction, not a self-authored one, so baking a self-merge step into a wakeup/loop prompt is fine for the rest of that session. See `mwc` for the grant's scope and limits.

In the **clear-all family** (`ardia`, `gia`, `gii`, `gip`), "report ready, don't merge" gates only the merge — it does **not** pause the sweep. A clean-but-unmerged PR is not a stop; move to the next item, and stack it when it isn't naturally independent of that PR. See `stack-dont-pause`.

The same gate does not pause the loop *within* a single PR either, and that is the harder half to see.

The tell is lexical, and it sits in your own outgoing message: a RECOMMENDATION or question whose proposed action is ordinary ARDI work.

- **Do:** resolve conflicts, sync, push fixes, and re-dispatch reviews on a PR whose merge you are correctly withholding, and report those in the past tense.
- **Do:** name the one action that is gated, so "blocked" stays a claim about a step rather than about the PR.

- **Don't:** generalize a withheld merge into withholding the rest of the loop as though one authorization covered both.
- **Don't:** write a recommendation proposing work ARDI already mandates — a request to do the required thing is the error, not a courtesy.

See `ardi.cases.md`, "A merge gate is not a work gate".

Self-review against the project's own stated conventions before every push, not just the first — and don't just re-read the criteria, actually run the applicable review skills against your own diff and iterate on what they find, the same ARD cycle you'd run against an external reviewer's findings. Don't treat the review bot as the mechanism that discovers a project's documented conventions — self-apply them first.

See `ardi.cases.md`, "A review round surfacing five findings your own conventions already covered".

Pre-push checklist

Pause point: after committing, before `git push`.

- ☐ **The whole test suite ran**, not the files you predicted the change touches, and the tests/failed/**skipped** triple was read — a non-trivial skip count means re-running with the gating flags set (`NOT_CRAN=true`, and whatever else un-gates a conditional skip).
- ☐ **Generated trees were regenerated** if the diff (or a `main` merge) touched a generator's inputs, and the PR body states how many changed files are generated.
- ☐ **Added lines were scanned** for banned punctuation and multi-sentence lines, run *after* committing, *after* every pass that edited the diff (your own reflow included), and with the three-dot range (`origin/main...HEAD`) — a pre-commit run reports on the wrong tree, a later edit retires the lines an earlier run scanned, and a two-dot range re-attributes whatever `main` deleted to you. The mirror direction fires at merge-decision time rather than at line-scanning time, and reads as an emergency: a two-dot `git diff origin/main` shows whatever `main` **added** as *your* deletions, so a behind branch appears to be reverting a sibling PR that just landed. It is not. A merge is three-way, so a file this branch never touched keeps `main`'s version, and a deletion count in an untouched file means the branch is behind rather than dangerous. Settle it with `git merge-tree --write-tree origin/main <head>` and read the resulting tree, rather than acting on the two-dot diff.
- ☐ **The changelog entry and EVERY PR description this round touched were re-read** against the new behavior, not just the code — none is in the diff, so no reviewer and no grep will catch a stale one. Read "every" literally: a round that corrects a claim appearing in two PRs' bodies discharges the *feeling* of having synced bodies as soon as one of them is done, and the one most likely to be skipped is your own, because fixing the other repo's copy is the part that felt like the work. This fires on a **prose** diff too: a body

that explains the claim the round just walked back is stale in the way that matters most, and "reconciling prose" does not feel like changing what the PR does. Every **number** in the body was re-*derived* by command rather than re-read, run *at this push* rather than carried from the last one, with the command pasted beside it — a wrong count reads exactly as plausible as a right one, so reading is no instrument for it, and a base figure owes its own derivation rather than riding on the delta's. "At this push" includes a push that only answers a self-review finding, and it includes the figures a "Corrections to this body" entry already refreshed — that entry is a claim about the previous head, so the current push is what expires it. A figure whose deriving command carries a **precondition** owes one more step, because deriving it freshly discharges "don't recall it" and says nothing about whether the command was right for this diff: cross-check it against a quantity computed by something else (`git diff --shortstat` against a hand-run added-lines count), since re-reading a correct-looking pipeline confirms it (`fail-fast`).

- ☐ **The diff's deleted lines were read** (`git diff origin/main...HEAD | grep '^-'`), and each one was a decision rather than collateral from an edit's blast radius — a reviewer reads every deletion as deliberate and will rationalize an accidental one.
- ☐ **`main` was merged in** if it moved, with version parity re-checked afterward, so the round costs one review run rather than two — and any whole-file count a merge can worsen (spliced changelog bullets) compared before against after, since a defect caused by a *deleted* line is invisible to every added-lines check (`sync-with-main`).
- ☐ **Killer item: the push landed.** `git rev-parse HEAD origin/<branch>` agree before any reply asserting a fix. This one is marked because its failure is not an omission but a **false claim about state**, which a reviewer has no reason to doubt: CI reports green because it correctly validated the older head, and the session's own recollection agrees with the reply. It answers whether the **branch** moved, and nothing about whether the **PR** is still open — a closed PR keeps accepting pushes and stops tracking its branch, so both SHAs agree while the PR's own head stays frozen. Read the PR's `state` as a second check, per `use-existing-pr-branch`, rather than letting this item stand for both.

Review a round's fixes as one diff, not as N independent fixes: two of them, each correctly addressing its own finding, can compose into a defect neither introduces alone.

- **Do:** re-read the round's full diff as a unit once every finding is addressed, before running the pre-push checklist.
- **Do:** treat a multi-item commit message as the prompt to check the items against each other.
- **Don't:** conclude the round is sound because each finding's fix is sound; that is a claim about the parts.
- **Don't:** rely on the next review round to catch it — it may, but it is then spending a round on something the previous round created.

See `ardi.cases.md`, "Two correct fixes composing into a defect neither introduces alone".

A clean verdict does not certify that your diff contains only what you meant, because a reviewer cannot tell an accident from a decision.

- **Do:** read your diff's deleted lines before pushing, and confirm each one was a decision rather than a casualty of an edit's blast radius.
- **Do:** say plainly, in the thread, when a review has blessed something unintended — the reviewer cannot know, and its verdict will otherwise stand as the record.
- **Don't:** treat a clean verdict as evidence about intent; it is evidence about correctness only.
- **Don't:** keep an unintended change because the reasoning offered for it turned out to be good.

When the edit is a regex or string patch rather than the Edit tool, two mechanisms turn that displacement into a silent over-deletion, and a self-check can wave both through.

- **Do:** anchor a string or regex patch on text unique to the intended site, and prefer the Edit tool's exact-string matching over a broad `.*?` DOTALL span.
- **Do:** make a patch self-check assert that neighbouring structures survive — the sibling function or loop still present, untouched element counts unchanged — not merely that the changed element's count is as expected.
- **Don't:** trust a `re.sub(..., flags=DOTALL, count=1)` whose non-greedy `.*?` start anchor is non-unique; it binds to the first occurrence.
- **Don't:** read "the assertions passed" as "the patch is correct"; a count-based check can pass by coincidental balance, and the `git diff` deletion-review is the real gate.

A clean verdict does not discharge the self-review against project conventions either, and the reviewer's own "not a finding" is where that shows up.

- **Do:** re-run the project-conventions check against your own diff after a clean verdict, not only before the push.
- **Do:** read a reviewer's "observations" and "not a finding" items as candidate violations, and grep `CLAUDE.md` for whatever they discuss.
- **Don't:** let a reasoned "belt-and-suspenders is fine" settle a question the repo already answered in writing.
- **Don't:** treat a non-blocking label as deciding whether an item gets checked at all.

Proactively self-correct a technical claim you already told a reviewer, the moment further testing shows it was wrong — don't wait for the reviewer to catch it. If you stated a rationale (an approach is safe, a risk doesn't apply, a backstop exists) and then discover through your own follow-up verification that it's false, post the correction with the actual evidence immediately, rather than leaving the stale claim standing until a review round re-raises it. This keeps the review loop converging instead of churning on a claim you already know is wrong.

See `ardi.cases.md`, "Self-correcting a rationale before the reviewer re-raises it".

A fix is not "pushed" until it is on the PR's head commit — verify with a SHA comparison before telling a reviewer you pushed it. From inside a session, an edited working tree and a pushed commit feel identical, so a round that edits the files, writes the reply, and never runs `git push` produces a reply asserting a fix that does not exist on the branch. Nothing contradicts it: CI reports green, because it correctly validated the older head; the next review round reviews code without the fix; and the session's own recollection of having made the change agrees with the reply. That makes it worse than an ordinary wrong claim — it is a false statement about *state*, which a reviewer has no reason to doubt and no cheap way to check.

A SHA you put in a PR body or a reply must be read, never recalled — and the PR body is where an invented one survives longest.

Knowing the prefix genuinely does not discharge this for the full SHA a link wants.

- **Do:** read every SHA you cite out of `git rev-parse` or `git log`, and confirm it resolves before pasting it.
- **Do:** correct a published wrong SHA with a visible note naming the real one.
- **Don't:** write a short SHA from recollection because it looks like the commit you just made.
- **Don't:** extend a genuinely-read short prefix into a full SHA by hand — the prefix discharges nothing for the 33 characters it does not contain.
- **Don't:** expect review to catch it — a reviewer has no reason to suspect a citation, and the body is not in the diff they are reading.

See `ardi.cases.md`, "A genuinely-read prefix, extended into a fabricated link".

The same rule governs a merge or squash commit message, which is worse than a PR body on both counts the bullet above names.

The trigger is a PR with no closing issue, which is why "verify identifiers" does not reach it.

An invented number here can close someone else's live work, because issues and pull requests share one number space.

- **Do:** read any `Closes`/`Fixes`/`Refs` number in a merge message out of the PR body it came from, and confirm the target is what you think it is.
- **Do:** say plainly that a PR closes nothing when it has no tracking issue, rather than leaving the slot empty for a number to fill later.
- **Do:** correct a published wrong reference visibly, in a comment, since the message itself cannot be amended once it is on the default branch.

- **Don't:** treat a closing keyword as inert against a pull request — it closes one, and the number space is shared with issues.
- **Don't:** infer from "nothing changed state" that a wrong reference was harmless; check whether the target was open.

See `ardi.cases.md`, "An invented `Closes` in a merge commit message".

A SHA's provenance is the question its source command answers, not merely that a command produced it.

- **Do:** name the claim, then run the one command that answers it, and paste that command's output.
- **Don't:** lift a SHA out of nearby command output because it is genuine and close at hand — `git stash list` and `git reflog` answer about their own subjects, not about the tip.

See `ardi.cases.md`, "A read SHA can answer a different question".

A verification table you write in the PR body is the same defect one artifact over, and re-reading it cannot catch a wrong number.

It goes stale rather than being wrong on arrival.

It sits in the PR body, which nothing re-reads.

- **Do:** re-derive every count in the PR body with a command at push time, and publish the command next to the count.
- **Do:** treat any round that changes the diff as expiring every figure the body already states, not only the figure that round was about.
- **Don't:** substitute re-reading for re-deriving — re-reading is the right instrument for a stale description and no instrument at all for a stale number.
- **Don't:** report a delta without deriving its base; a base carried from recollection is unfalsifiable by any later check of the delta.

See `ardi.cases.md`, "A verification table in the PR body going stale as rounds change the diff".

A reviewer's round-one confirmation of that table does not expire when the diff moves, and the confirmation is what makes the stale figure dangerous.

The rule above says the figures go stale. This is about the one artifact in the review record that argues they have not.

Round 1 verifies the table, in detail, because a body full of derived counts is exactly what a first review checks, and it says so, naming each figure it matched. Round 2 does not re-verify it, because round 2 is not about the table. Round 2 is about whether round 1's findings were addressed, so the body sits outside what that round set out to read.

The confirmation is therefore a claim about one head, and nothing retires it. An unverified table at least invites suspicion. A table a reviewer explicitly confirmed reads as settled by someone other than its author, and that reading survives every push that falsifies it.

Sharper still, and this is the part worth pinning: round 2 can derive the correct new figure and use it in its own prose while the body carries the old one, and flag nothing. The reviewer is not diffing its numbers against the body's. The reviewer derives fresh ones for its own purposes, so the two figures sit one round apart in a single comment thread, contradicting each other, with nobody comparing them.

- **Do:** re-derive every figure in the body at each push, whatever an earlier round confirmed, and record the SHA the new figures were derived at.
- **Do:** compare any figure a later review states in its own prose against the figure the body states, and read a mismatch as the body being stale.
- **Don't:** carry a round-one confirmation forward to a later head — it verified the diff that existed when it ran.
- **Don't:** read a later round's clean verdict as evidence the body is still accurate; that round checked the findings, not the table.

See `ardi.cases.md`, "A round-one confirmation laundering a body the next round contradicts".

A "Corrections to this body" entry is itself a figure in the body, so the next push expires it too — and it reads as more settled than the figure it corrected.

- **Do:** re-derive every figure a corrections entry vouches for at each push, and record the SHA the new figures were derived at alongside them.
- **Do:** append a further numbered entry when a later push moves the figures again, rather than editing the previous one, so the round that expired them stays visible.
- **Don't:** read a corrections entry as discharging the figures it names — it is a claim about one commit, and the next push is what falsifies it.
- **Don't:** treat having written the correction as having done the check; the note is that check's output, never a substitute for re-running it.

See `ardi.cases.md`, "A corrections entry expires with the next push".

Verifying that a stale figure is gone needs a SECTION-scoped search, because the corrections entry legitimately quotes it.

The two rules above compose into a check that cannot discriminate. The table must stop claiming the superseded figure, and the corrections entry must quote that same figure in order to say what changed — so the string is still in the body after a fully correct fix, and a whole-body search for it reports that fix as having failed.

That is a check whose pass path and failure path look alike, which `fail-fast` says is not yet a check. It also fails in the direction that invites damage: the natural response to a "still present" hit is to delete the quotation, which is the one part of the entry carrying the record.

Scope the search to the section that makes the claim, and assert the corrections entry in the opposite direction:

```
ver  = body[body.find("## Verification"):body.find("### Corrections")]
corr = body[body.find("### Corrections"):]
assert "484 added" not in ver    # the table no longer claims it
assert "484 added" in corr       # the entry still records what changed
```

- **Do:** scope a staleness check to the section that makes the claim, and assert separately that the corrections entry still quotes the old figure.
- **Do:** write the two assertions in opposite directions, so a deleted quotation fails as loudly as an uncorrected table.
- **Don't:** search the whole body for the superseded figure — a correct fix leaves it present, so that search reports every correct outcome as a failure.
- **Don't:** answer a "still present" hit by removing the quotation from the corrections entry; that quotation is the record the entry exists to carry.

See `ardi.cases.md`, "A whole-body staleness check that reported a correct fix as failed".

The one case where a figure does **not** expire is a push that leaves the tree unchanged — a revert-and-restore returns the tree to an object it already had, and a measurement is a function of the tree rather than the commit. `dont-incur-technical-debt`'s "The one exception" section carries that mechanic, and the deferral it licenses.

The read side of that comparison can lag a push by a few seconds, so test the two *local* refs against each other before concluding anything failed.

- **Do:** compare `HEAD` against `origin/<branch>` first when the PR API disagrees, and re-read rather than re-push when those two agree.
- **Don't:** amend, force-push, or re-commit on the strength of an API SHA alone.

A brand-new branch can read back at the wrong commit, so the local two-ref comparison above is not sufficient there.

The gap is in the trigger rather than in the remedy.

The likeliest explanation is a local one, and it reproduces offline.

See `ardi.cases.md`, "A brand-new branch reading back at `main`'s tip, reproduced offline".

- **Do:** run `git ls-remote origin <branch>` after the first push to a new branch, and compare its SHA against `git rev-parse HEAD`.

- **Do:** run `git rev-parse HEAD <branch>` first when those two disagree, since a branch ref left behind accounts for the whole signature (and note that `--short` rejects a second revision, so pass neither).
- **Do:** re-run plain `git ls-remote` as well, so a ref that self-corrects stays distinguishable from one a re-push repaired.
- **Do:** re-push with `git push origin HEAD:refs/heads/<branch>` when the mismatch persists, and read the SHA range it prints as the confirmation.
- **Don't:** treat a `git push` that exited 0 and printed `* [new branch]` as evidence the commit reached the remote.
- **Don't:** assume `git push -u origin <branch>` sent the commit you just made – it sends the branch ref, which `HEAD` may have moved past.
- **Don't:** credit a corrective re-push with having repaired a remote-side fault when neither of those two controls was run.
- **Don't:** answer a `No commits between main and <branch>` error by re-checking the base branch argument before checking where the head ref actually points.

The same false claim arrives as *incoming* state when you pick a PR up mid-flight, and there the SHA comparison usually has nothing to compare.

- **Do:** run `gh pr diff <N> --name-only` against any inherited "already fixed" claim before deciding a finding is closed.
- **Do:** state plainly in your own summary that the prior claim did not hold, and name the head it was false at.
- **Don't:** treat green CI as evidence that a claimed fix landed.
- **Don't:** infer that a finding is stale because a comment says it was addressed.

Run that same command before *any* readiness claim, not only against an inherited one — a PR whose branch carries no implementation is green on every check.

- **Do:** run `gh pr diff <N> --name-only` before reporting a PR ready, and read the returned paths against what the PR says it does.
- **Do:** treat an empty return, or a return holding only a `main` merge's incidental paths, as the PR carrying no implementation.
- **Don't:** count the claim commit or a `main` merge as work — neither is implementation, and both give the branch a plausible history.
- **Don't:** read all-green CI plus a finding-free review as evidence a PR contains anything; on an empty diff that is the expected result.

When the change affects downstream consumers, validate it against a real consumer repo before reporting the PR ready — a package's own test fixtures are built to exercise its code, not to resemble the packages that will actually use it.

- **Input shapes no fixture happens to contain.** A real package carries metadata the fixtures never needed — an entry of a different kind, an extra tag, an unusual name — so a branch written for it has never actually run on real input.

- **Message formatting under real counts.** Fixtures usually trip the plural path; a real repo hitting the same code with exactly one item exercises the singular wording, which no test asserted.
- **The migration/upgrade path, as opposed to the fresh-install path.** This is the one fixtures can never reach: a fixture is created new by the test, so it always gets the current templates. An existing consumer has the *old* config, and whether the feature reaches it at all is a different question from whether it works. Verify the claim in the changelog by running the documented migration step, rather than describing it.

See `ardi.cases.md`, "Validating against a real consumer repo covers what fixtures cannot".

Verify a blocker you assert in a PR body or a reply, with the same rigor you apply to a reviewer's claims — a stated blocker becomes a premise other people build on.

Attempting the base form of a command is not attempting its variants — a refusal describes the invocation you ran, never the flag you did not try.

- **Do:** run the flag variant the docs or the error itself name, before generalizing a refusal into an impossibility.
- **Do:** scope the published claim to the invocation actually run, naming the exact command and what it exited with.
- **Don't:** read an unconditional-sounding error as covering flags you never passed.
- **Don't:** count an attempt at the base form as discharging the rule above for a variant of it.

Name the specific gate when you report a blocker, not a category word that happens to be one of several.

- **Do:** quote the clause that distinguishes the failure, and name the gate it belongs to.
- **Do:** re-read a blocker you have restated several times, since a paraphrase repeated across status reports hardens into the record.
- **Don't:** use one mechanism's own name as a generic word for its category.
- **Don't:** treat having verified *that* something is blocked as having verified *why*.

When the blocker is a hang, inspect the process rather than re-guessing what it is waiting on.

- **Do:** read `ps -o stat=`, `lsof -d 0`, and the process tree before describing what a hung command is waiting for.
- **Do:** say which read produced the answer, so the gate is checkable rather than asserted.
- **Don't:** substitute one guessed mechanism for another because a probe produced no output.
- **Don't:** report a timeout signal as evidence about *why* something blocked; it is evidence only that it had not finished.

A blocker that was true when you published it can stop being true while the PR is open, and withdrawing it is your job, not the reviewer's.

- **Do:** after every `main` merge, scan the PR's touched files for merge-status hedges with whitespace-normalizing search, then re-read each hit against the new base.
- **Don't:** assume a hedge survived because the file that contained it merged without conflicts, or because literal grep missed a phrase split across semantic lines.

Landing a fix falsifies whatever prose documented the defect, and that prose is never in your diff — so grep for it rather than expecting to be reminded.

- **Prose staled by the fix.** It was accurate when written, so nothing about it reads as a defect, and a workaround it prescribes becomes active misdirection the moment the thing it worked around is gone. Keep the entry where the old behaviour explains something — most of a corpus is written against it — but mark plainly that it is history and name the change that ended it.
- **Prose asserting conformance to a reference.** A docstring saying the code "follows" some reference implementation is a claim about two artifacts, and your own divergence falsifies it. This one is not staleness at all: it was false before you arrived, and it is load-bearing, because a reader checking the code against the reference stops at the sentence saying someone already did.
- **Do:** grep the repository for the defect, the workaround, and the behaviour you changed, before calling a fix complete.
- **Do:** mark a superseded entry as history and name the change that ended it, rather than deleting it, when the old behaviour still explains other text.
- **Don't:** treat a clean grep over the diff as coverage — the stale prose is outside it by construction.
- **Don't:** leave a doc asserting conformance to a reference standing when the code diverges; correct the claim in the same change that establishes the divergence.

An instruction's own suggested code is not exempt from the project-conventions self-review above.

See `ardi.cases.md`, "An instruction's own suggested code breaking a project convention".

When the code path under test has a staging or transform step between input and output, a passing unit suite is not evidence it works — exercise the real path once.

See `ardi.cases.md`, "A staging step the unit fixtures could not reach".

When new code branches on a third-party tool's behavior, read that tool's own config or docs for the specific behavior — don't infer it from what the tool broadly does.

A regression test written alongside a fix can lock the bug in rather than catch it — assert the two paths that diverge, not the one you just touched.

A systematic audit done by skimming is worse than the one-at-a-time version it replaces.

Adding an explanation supersedes whatever the file already said about the same thing, so re-read the older passage — your own diff is the likeliest source of a contradiction nobody flags.

The same rule applies within a single diff, and there nothing prompts the check at all.

And when the explanation you add is a *mechanism* claim, test the class it distinguishes, not just the sample in front of you.

See `ardi.cases.md`, "A mechanism claim whose population held no true positive".

A symptom that stops reproducing is a fix having landed, until you have checked otherwise — reaching for nondeterminism is the attractive wrong answer.

- **Do:** look for a merged fix, and date it, before attributing a vanished symptom to anything.
- **Do:** report the before/after with its timestamps, so the negative control is visible rather than asserted.
- **Don't:** explain a symptom's disappearance as nondeterminism on the strength of one clean run.
- **Don't:** carry such a claim into an issue or a decision doc, where it argues against the very fix that produced the silence.

The mirror runs the other way, and it is the one that discards a good fix: a symptom that KEEPS reproducing after a fix landed is not evidence the fix failed.

The bullet above governs a symptom that vanished, where the attractive wrong answer is nondeterminism. Here the symptom is still there, and the attractive wrong answer is that the diagnosis was wrong — which sends you back to re-litigate a fix that is working, and leaves the real remaining cause unread.

The mechanism is ordinary and worth naming, because it makes the persistence expected rather than surprising. A failure can have causes in series, and only the first one is observable while it stands. Removing it does not change the outcome; it changes which cause produces the outcome. So the job's colour is the same before and after, and the outcome is the one thing everybody checks.

The discriminator is the error, not the outcome. Both runs failed, so comparing conclusions establishes nothing. Comparing the error text is decidable in one read, and a changed error means the first cause is gone and a second was behind it. Where the fix is upstream, pin the comparison to the dependency version each run actually resolved, since a run predating the fix is not evidence about it — `dont-reinvent-wheel`'s "mirror direction" section owns that lookup.

Note the asymmetry that makes this worth a rule. Reading the new error costs one glance and usually names its own remedy. Re-litigating the first fix costs a round, and it argues for reverting something correct — the same shape the bullet above warns about, where a claim ends up arguing against the fix that produced the change.

- **Do:** diff the error text across the fix, not the pass/fail outcome, before concluding anything about whether the fix worked.
- **Do:** resolve which dependency version each run used, when the fix landed upstream, so a pre-fix run is not read as evidence against it.
- **Do:** report a changed error as a second cause found, and file it, rather than as the first fix having failed.
- **Don't:** re-open a landed fix because the symptom persists — that is a claim about the outcome, and the outcome is what a serial second cause preserves.
- **Don't:** read the earlier bullet as covering this; it fires on a symptom that stopped, and this one fires on a symptom that did not.

See `ardi.cases.md`, "A trust-gate fix that revealed a tool-name mismatch behind it".

Verify a command, path, or flag *you* write into a doc, with the same rigor `address-every-comment` demands for one a reviewer suggests.

- **Do:** confirm every literal you invent against the tool's own source or help output before it lands in a doc.
- **Do:** cite the file or command you checked, so the claim stays falsifiable.
- **Don't:** infer a subcommand from a family that has its siblings (`gh label list`/`create`/`edit` does not imply `gh label view`).
- **Don't:** treat a literal as exempt because the prose around it is well-sourced — the literal is the part a reader executes.

Run that check over your own fix, too — the remedy for an unverified literal is where the next unverified literal goes.

- **Do:** re-run the rule you are applying against the text of your own fix, before committing it.
- **Do:** say in the thread when a fix's own draft tripped the same rule, since that is the only place the near-miss is visible.
- **Don't:** treat the effort of writing a correction as evidence the correction is verified.

The same rule reaches past a literal, to the defect CLASS a code fix just closed.

A consolidation commit is the highest-risk host for it, and the likeliest to be trusted.

- **Do:** ask whether the fix's own new code instantiates the class it closed, before committing it.
- **Do:** treat a commit that consolidates one duplicated concept as owing a check that it forked none, since its framing argues the other way.
- **Do:** compose an existing shared anchor or helper into a new site rather than hand-rolling an equivalent, so the site inherits later fixes too.
- **Don't:** read a diff that removes a duplicate as evidence that it added none.
- **Don't:** treat the next round's finding at a new address as a fresh gap without first checking whether your own previous fix created that address.

When regenerating a generated tree makes it most of the diff, say so in the PR body — otherwise a reviewer reads it as pollution and blocks.

- **Do:** grep a file for a generated-by header before editing it, and change the source instead.
- **Do:** state in the PR body how many of the changed files are generated, and name the hand-written ones.
- **Don't:** revert generated output because a reviewer calls it noise — check first whether the sync check requires it.
- **Don't:** assume a reviewer sees the source files; on a large diff they frequently do not.

See `ardi.cases.md`, "Editing generated output, then being read as pollution once regenerated".

Run the whole test suite before pushing, not the files you predict the change touches — and check that the ones you ran were not silently skipped.

Matching the tool's VERSION is not matching its ENVIRONMENT, and when the tool GENERATES a file you are about to commit, the gap ships.

Read the generator's own diagnostics first, because a good one says so outright and names the cause.

The file list is the backstop, for a generator that degrades with no diagnostic, or one whose diagnostics scroll past in a long run.

- **Do:** read the generator's own warnings before its output — roxygen2 names the missing package and the tag that needed it.
- **Do:** compare your generator's changed-file list against the CI log's, and treat any extra file as an environment mismatch until explained.
- **Do:** install the optional/dev dependency set as well as the tool, when a generator loads the package to do its work.

- **Don't:** read "I installed the same version CI installs" as having matched CI — version is one input to the output, and rarely the one that differs.
- **Don't:** commit generated output whose diff is wider than the job you are trying to satisfy reported.

See `ardi.cases.md`, "A generator's environment, not its version, changed the committed artifact".

- **Do:** run the full suite before pushing, and state the tests/failed/ skipped triple rather than "tests pass".
- **Do:** set the flags that un-gate conditional skips, and re-run if the skip count is non-trivial.
- **Don't:** scope a local run to the files you edited — the test asserting the old behaviour is usually somewhere else.
- **Don't:** read a green subset as a green suite, or a skip as a pass.

Running a script is not running its tests, and an "advisory" check can have a hard-gating twin.

- **Do:** run every check the CI job runs, its test files included, before pushing.
- **Do:** grep the job definition for other steps touching the same property before saying anything about whether it gates.
- **Don't:** substitute a production script's exit code for its test file.
- **Don't:** infer a job's behaviour from one step's label — "(advisory)" describes that step, not the job.

A third failure mode of the whole-suite rule above: the suite holds no case that could have failed.

- **Do:** construct the input class the change is supposed to handle and diff its behaviour against the pre-change code, before calling a guard verified.
- **Do:** name which cases could have exercised the defect class, rather than quoting a suite total — the total is a fact about the suite, not the diff.
- **Don't:** offer a pre-existing suite's green as verification of a change it holds no case for; those cases predate the defect and cannot speak to it.
- **Don't:** read the tests/failed/skipped triple above as covering this — it makes the report more precise without making it any more relevant.

A fourth failure mode: the case exists, and which branch it reaches is decided by the host.

- **Do:** name in the test which host-derived value selects which branch, and add a case that pins each branch regardless of that value.
- **Do:** run a host-dependent suite in both environments before believing its coverage, and say which branch each run took.

- **Don't:** read green in CI as covering a branch whose selection depends on an input CI happens to supply one way.
- **Don't:** reach for the skip count here — nothing is skipped, so that component is identical on both machines even when the failed counts diverge.

See `ardi.cases.md`, "A suite whose branch coverage varies by host".

What "Fully Clean" Means

"Fully clean" is the terminal state the ARDI review loop drives toward. A PR/MR is **fully clean** when **both** of these hold (and verified via `python3 scripts/check-pr-fully-clean.py <pr-number>`):

Extended rationale — the mechanism, evidence, and argument behind each rule below — lives in `fully-clean.rationale.md`, moved out of the auto-loaded context. Each rule here keeps its statement and its Do/Don't pair; read the companion when the reasoning or the evidence is the question.

Worked-example case records for the rules below live in `fully-clean.cases.md`, moved out of the auto-loaded context.

1. **All CI workflows and check runs are green AND completed.** Every workflow and check run passes — not just the required checks and not just the review job.

   `status` itself can be stale, so never infer a job's *duration* from it.

   - **Do:** read elapsed time from log timestamps whenever the length of a run is the thing being judged.
   - **Don't:** conclude a job is still running, or has passed some duration threshold, from `in_progress` plus the wall clock.

   When you are waiting for a job rather than timing one, poll its step list instead of its status — the steps are not subject to the same lag.

   - **Do:** poll `actions/jobs/<id>`'s `steps[]` when waiting on a specific job, and treat its terminal step completing as the signal.
   - **Do:** report which step the job is on, so a stalled job is distinguishable from a slow one.
   - **Don't:** poll a check run's `status` in a loop and read repeated `in_progress` as evidence the job is still working.

   See `fully-clean.cases.md`, "Poll a job's step list, not its check-run status".

   A `BlobNotFound` / HTTP 404 on the job-log fetch means the job has not completed, not that it has hung.

- **Do:** read a 404 / `BlobNotFound` on the job-log endpoint as "the job has not finished", and wait for completion (or read the live UI log) before judging its outcome.
- **Do:** take a job's real state from its `status`/`conclusion`, since the same 404 covers a still-running job and a completed-with-no-logs one.
- **Don't:** read a 404 on the log fetch as positive evidence of a hang or a stall — it is the opposite, evidence the job is still running.
- **Don't:** file an issue reporting a review job as hung or "no verdict produced" while its log fetch still 404s and its status is `in_progress`.
- **Don't:** run the rule backwards: a log URL being **served** is not evidence the job completed. The blob can exist mid-run, so a successful log fetch and a still-running job coexist. Completion comes from `status`/`conclusion` alone, in both directions.

`gh pr checks` is not a complete enumeration of a head's check runs, so read the commit check-runs endpoint before deciding that everything has finished.

`--paginate` is load-bearing, not tidiness.

The endpoint covers check runs only, so a repo that still uses legacy commit statuses needs a second query.

Why the two surfaces disagree is unexplained, so do not assert a mechanism for it.

- **Do:** take the check-run half of criterion 1 from the paginated check-runs endpoint, and add `commits/<sha>/status` where the repo uses commit statuses, rather than treating either query as sufficient alone.
- **Do:** report both counts when the endpoint and the rollup disagree, so the gap stays visible to whoever reads the status next.
- **Don't:** read `0 pending` from `gh pr checks` as evidence that nothing is still running.
- **Don't:** drop `--paginate` — an unfinished run on page 2 returns the same empty result as a finished head.
- **Don't:** offer a reason for the omission — none was established.

A check-run NAME is not unique across workflows, so a name alone does not identify which check passed. Two workflows in one repo can each define a job with the same name, and `gh pr checks` prints the bare name with no workflow attached — so a passing row can belong to a workflow you were not asking about. The ambiguity is invisible in the output, which is what makes it dangerous: nothing in a duplicated name looks different from a unique one, so no prompt to check ever arrives.

Resolve it from the run behind the check rather than from the name:

```
gh api "repos/<owner>/<repo>/commits/<sha>/check-runs" --paginate \
  --jq '.check_runs[] | select(.name == "<name>") | .html_url'
gh run view <run-id> -R <owner>/<repo> --json workflowName --jq .workflowName
```

Cross-check against the workflow's own job list too. A matrix leg gated on `needs:` may not have started at all, so its absence from a run's jobs contradicts any same-named row reported as passing.

`check-pr-fully-clean.py` annotates a duplicated name with the run URL only on the lines it actually reports — a run still pending, or one that finished badly. A **passing** duplicated name produces no line at all, so it is never annotated, and the manual lookup above is the only thing that resolves it. That is precisely the case this section was written from: the passing row belonged to the wrong workflow, and nothing in the script's output would have said so.

- **Do:** take the workflow from the check run's own URL before attributing a pass or a failure.
- **Don't:** read a job name as identifying a workflow — it identifies a job, and two workflows may define the same one.

(Measured 2026-08-21 on `ucdavis/bcs`: `ubuntu-latest (release)` exists in both `R-CMD-check.yaml` and `check-readme`. On a PR fixing an `R CMD check` failure, the passing row was `check-readme`, while `R-CMD-check.yaml`'s matrix legs had not started — they are gated on `needs: [matrix, update-snapshots]`. Reporting the regression fixed on that row would have cited an unrelated workflow.)

Every subsection above explains a check list that is short for a per-PR reason, and a platform outage produces the same shape for a reason none of them can reach.

A job's conclusion is set by whichever step failed, which need not be the step whose verdict you read. Every rule above is about an enumeration that came back short. This one is the opposite case: the enumeration is complete and terminal, and the answer you read came from the wrong member of it. A workflow can carry a guard step that decides what a run *meant* — a review guard classifying an outcome, a summarizer, a status resolver — and that step can conclude "this is fine", write its output, and end `success`, while the job is red because an earlier step failed without `continue-on-error`. Reading the guard's own log line then reports the opposite of the check. So when a red job's log carries a green verdict, do not treat it as a contradiction to explain: enumerate the steps and find the one whose conclusion is `failure`. The same reading also settles what to do next: whether a fix to the classifier can clear the check at all, since a classifier the job does not consult is fixable without changing anything the reader sees.

- **Do:** identify the failing *step* before diagnosing a failing job, rather than reasoning from whichever step's output you happened to read.
- **Do:** treat a green guard step beside a red job as evidence about the wiring, since the two were decided by different steps.
- **Don't:** read a guard step's own log line as the job's verdict — the two are decided by different steps, so agreeing is a coincidence rather than a confirmation.

- **Don't:** claim a fix to a classifier clears a check until you have confirmed the job's conclusion actually depends on that classifier.

See `fully-clean.cases.md`, "A green guard step beside a red job".

One SHA can carry two check runs of the same name, from the same workflow, with opposite conclusions — because a workflow gated on a base-ref diff runs VACUOUSLY on `push` and meaningfully on `pull_request`. The subsection above covers a green step inside a red job. This is the mirror at the run level, and it is worse, because nothing about the green one looks partial: it reports the same check name, it completed, and it passed.

The mechanism is a workflow that needs a base to diff against. `check-new-line-breaks.yml` passes `base-ref` only when `github.event_name == 'pull_request'`, so the `push`-triggered run of the identical workflow has no base, examines zero added lines, and passes having measured nothing. Both runs attach to the same commit, so `gh pr checks` prints two rows with one name, one `pass` and one `fail`, and reading the list top-down finds whichever came first.

The vacuous run is the one to discard, and the trigger event is the only field that separates them. `gh api "repos/<owner>/<repo>/actions/runs/<id>" --jq '.event'` settles it in one read per run. A `pass` from a run whose event supplies no base is `batch-merge-and-resolve`'s zero-matrix problem arriving as a green check. That fragment states it as "a matrix of zeros is indistinguishable from a detector that never ran", and prescribes a negative control before trusting any zero. The same remedy applies here, and the trigger event is what supplies it: a run given no base examined nothing, so its `pass` is the zero rather than a result.

Note that this is not the same as ai-config#1870's ambiguity, where two *different* workflows contribute check runs sharing a name. Here it is one workflow, and the disambiguator is the event rather than the workflow name — so a fix keyed on `workflowName` cannot see it.

- **Do:** read the `event` of any run whose verdict you are about to rely on, whenever the same check name appears twice on one head.
- **Do:** take the verdict from the `pull_request`-triggered run for any check that diffs against a base.
- **Don't:** read a `pass` as evidence the check examined anything — ask what population it was given first.
- **Don't:** resolve a same-name disagreement by workflow name. On this shape both runs carry the same one.

(Measured 2026-08-22 on ai-config#1884. Run `32545283504` (`event=push`) and run `32545289903` (`event=pull_request`) both had `head_sha=8c456074`, both were named `new-line-breaks / check-new-line-breaks`, and they concluded `success` and

`failure` respectively. The push run was read first and taken as the verdict. The PR run was the one carrying four real findings.)

2. **The latest review is totally clean:** no nits, and every item that wasn't directly **Addressed** is either **Deferred** to a tracked follow-up issue, or **Rebutted with a rebuttal that actually convinced the reviewer** — i.e. the reviewer did *not* re-raise it on the next round.

Criterion 2's test is the absence of findings, not the presence of a verdict line saying so.

So when the two disagree inside one comment, **the findings win**. Read to the end of the comment before calling anything clean, and count the items under every heading, whatever that heading is called — `address-every-comment` already establishes that "non-blocking", "nit", "minor", and "optional" are prioritization labels rather than a pass, and a reviewer files findings under exactly those words in the section that contradicts its own verdict line.

Final approval comes from Claude where Claude is reachable. Another agent's clean verdict clears CI's review gate; it does not clear criterion 2 on its own.

This is a directive rather than a derivation, so treat it as a standing preference and not as a claim about any agent's general competence. What it settles is which verdict a PR is reported **ready** on.

The reason it needs stating is that the two are indistinguishable from the PR page. Every agent posts the same shape — a summary, some analysis, a positive closing line — so a findings-free report reads as approval whichever agent produced it, and the review-gate check goes green either way.

Two failure modes make the preference concrete, and both have recurred:

- **A clean verdict over tooling that errored.** A report can open by saying its own grep failed and then approve on the strength of the analysis that grep was supposed to support. The error line sits above the verdict, so it reads as a caveat rather than as the verdict's foundation collapsing.
- **A clean verdict at a head another agent finds a real defect in.** Not a difference of opinion about a nit — a checkable factual error, at the same commit, that the clean verdict passed over.

So when Claude is reachable, its verdict is the one to report on:

- **Do:** dispatch a Claude review and wait for its verdict before reporting a PR ready, whatever another agent has already said.
- **Do:** name which agent produced the verdict you are reporting, so "clean" is attributable rather than anonymous.
- **Do:** treat another agent's findings as real findings — this ranks whose *approval* is final, not whose objections count.

- **Don't:** report a PR ready on a non-Claude clean verdict while Claude is reachable, however thorough that report reads.
- **Don't:** read a green review-gate check as settling this; the gate does not know which agent answered, and on a selector-based setup the agent is chosen at random.

This is a different question from how much two reviewers **agreeing** is worth, which `self-review-fallback`'s cross-vendor section settles: there, same-vendor agreement measures a shared blind spot, and a cross-vendor split is a prompt to check the item yourself. That section weighs corroboration; this one names whose approval is terminal. They compose — a cross-vendor reviewer is still worth chasing, and its clean verdict still is not the one a PR is reported ready on while Claude is reachable.

Where Claude is genuinely unreachable — quota-skipped, a stub with no stated verdict, or not configured — fall back per `self-review-fallback`, which already governs that case. Another agent's clean verdict is worth more than nothing there, and it is still not Claude's; say which one you have.

See `fully-clean.cases.md`, "Two agents, one head, opposite verdicts".

Both criteria are per-PR, and a stack is where that stops being automatic.

- **Do:** derive a verdict per PR number, and name the PR beside each one.
- **Do:** treat a refusal from one reviewer on one PR as evidence about that reviewer on that PR, and nothing else.
- **Don't:** report a stack's review state from a single read — "I read the review" is a per-PR claim, and the stack is what makes it read as a claim about the work.

See `fully-clean.cases.md`, "Both criteria are per-PR, and a stack is where that stops being automatic".

The disagreement is measurable, and it is not a wording problem.

A reviewer's own verification block can be wrong while its verdict is right.

- **Do:** re-derive a posted verification's groups, not just its total.
- **Do:** fix the wording that invited a wrong reconstruction, even when nothing in the diff was false.
- **Don't:** let the word "verification" stand in for having verified.
- **Don't:** read a table that sums as one that partitions correctly.

A clean verdict can ratify an enumeration instead of testing it, and then it reads as independent corroboration of a false scope claim.

- **Do:** derive any enumeration you publish with a command, and publish the command beside it.
- **Do:** treat a reviewer restating your count as that count still being unverified.

- **Don't:** read a clean verdict as evidence that a scope claim in the diff is complete — a reviewer can only check the members you named.
- **Don't:** count a reviewer's agreement as independent when its population came from your own prose.

What "an approving review" means here is not a review state.

- **Do:** read the whole review comment and count findings under every heading before calling a PR clean.
- **Do:** establish approval from the findings and thread lists, since `.state` is `COMMENTED` on every review this repo receives.
- **Don't:** quote a **Ready for merge** line as the clean signal while the same comment lists findings.
- **Don't:** wait for a formal `APPROVED` review, or read `COMMENTED` as a defect in the reviewer.

Findings hide on several surfaces, and no single check sees all of them — so read the verdict body, any suppressed-comments block, the inline comments, the thread list, and the verdict's own conclusion every round.

- **An out-of-diff finding never becomes a thread.** A finding about a line the diff did not touch cannot be attached as an inline comment, so it appears only in the body — reviewers say so explicitly ("inline comments were unavailable for out-of-diff lines"). A thread count therefore cannot see it. Zero unresolved threads is not evidence of zero findings.

- **A notification that truncates the body hides exactly that finding.** The rule above says to read the body, and assumes you are reading the body. A CI-monitor or webhook event delivers the review as *quoted text*, capped at some length, and the inline findings are enumerated first because they are numbered — so what gets cut is the tail, which is where an out-of-diff finding and the verdict both live. The event is honest about it, and that is the trap: it prints a marker like `[truncated --- full text: gh api repos/<owner>/<repo>/issues/comments/<id>]`, which reads as a courtesy rather than as an instruction, and the visible portion looks like a complete, well-structured review. Acting on the inline comments alone then feels like having addressed the round, and the thread sweep confirms it, because the missed finding was never a thread. So run that command before treating a finding list as complete, whenever the review reached you through a notification rather than through a direct read.

- **An empty body hides the mirror case.** A review can post a completely empty top-level body and carry its entire finding in one inline comment, so a body-only read finds nothing to act on and concludes there is nothing.

- **A clean overview can hide a collapsed findings block.** Copilot can say it "generated no new comments" and create zero inline comments while placing substantive

findings inside a collapsed `<details>` suppression block in the review body. Match case-insensitively on `suppressed` **inside the `<summary>` heading**, not anywhere in the body. See `fully-clean.cases.md`, "The collapsed-block case (Morrison-Lab/ai-config#1029)".

- **"No verdict" is its own state, distinct from "a verdict with no findings".** A review job can fail having posted *nothing* — not a stub, not an empty comment. Zero findings and zero review are indistinguishable by any count, and they call for opposite responses: one is done, the other needs a self-review and a re-run. Read the job's step outcomes when a review is missing rather than inferring from the absence of comments.
- **The notification that wakes you carries a SUBSET of the findings, and nothing in it says so.** Every case above is a surface *on GitHub* that a query can reach. This one is the channel that tells you to look in the first place: a `pull_request_review_comment.created` wake delivers **one** comment, and a review posting five of them wakes you five times, asynchronously, with no count and no "1 of 5". So the first wake is indistinguishable from the only wake, and acting on it reads as responsive while leaving the rest unaddressed. It is worse than an ordinary partial read because the thread then *looks* handled: a reply and a resolved thread sit under the one finding you saw. Re-fetch `get_review_comments` on every review wake and act on the whole set, never on the wake's own payload.
- **Do:** read all review surfaces before calling a PR clean, every round, including collapsed suppressed-comments blocks.
- **Do:** distinguish "no findings" from "no verdict" explicitly, and treat the latter as unreviewed.
- **Don't:** report clean on a zero thread count, however many checks are green.
- **Don't:** treat an empty review body as an all-clear without checking the inline comments.
- **Don't:** treat a "generated no new comments" overview as an all-clear until every `<summary>` heading has been checked case-insensitively for `suppressed` — not until the whole body has, which flags ordinary overview prose that merely mentions suppressed findings.
- **Don't:** read a reviewer's silence as a verdict — a job that posted nothing leaves the same zero counts as a job that found nothing.
- **Don't:** act on a review wake's own payload — it is one comment out of however many the round posted, and it never says which.

A comment can be evidence-dense, correct throughout, and state no verdict at all — and its density is what gets read as the conclusion.

A later comment stating no verdict does not supersede an earlier one.

- **Do:** identify the last statement that actually states a verdict, and treat that as the standing one.
- **Do:** scan the whole review history for it, not only items matching HEAD.
- **Don't:** read a verification section, however rigorous, as an approval — it is evidence, and a verdict is a conclusion about evidence.
- **Don't:** treat a later comment's silence on the verdict as superseding an earlier "Needs more work".

See `fully-clean.cases.md`, "A later comment stating no verdict does not supersede an earlier one".

A reviewer skip notice (e.g. for workflow edits or quota exhaustion) does NOT clear or supersede prior review findings.

When a review run skips (e.g. self-modification workflow guard or quota limits) and falls back to a self-review or human review per `self-review-fallback`, that fallback authorizes **merging** only in the absence of prior unresolved findings. It does NOT wipe the slate clean, and it does NOT license merging over an unaddressed `Needs more work` verdict or open finding list from an earlier or concurrent review run.

- **Do:** scan the complete PR review comment history for any `Needs more work` verdicts or open finding sections before declaring a PR clean or ready to merge.
- **Do:** address, rebut (with convincing acceptance), or defer every previously raised finding even if the most recent review run skipped.
- **Don't:** treat a reviewer skip notice or self-review fallback as an all-clear or as permission to ignore open findings on the PR.

Another surface, and the one that defeats the gate itself: the review check can pass on a blocking verdict.

- **Do:** grep the verdict body for its own conclusion, and treat a `require-review` pass as orthogonal to whether the PR is clean.
- **Don't:** let a green review-gate check stand in for reading what the review said.

`check-pr-fully-clean.py` itself has the mirror false positive: it can report NOT clean over a clean verdict.

- **Do:** read the verdict's own conclusion when the script reports findings against a review whose prose merely discusses finding vocabulary.
- **Don't:** treat a `contains findings (matched pattern ...)` line as a real finding without reading the verdict body it matched.

Calling the checker is not consuming it: grepping its PROSE instead of reading its EXIT STATUS re-opens the whole failure one layer up.

The rule above and `no-handrolled-verdict-parse.py` both govern *bypassing* the instrument. This is the case where you run it, correctly, on the right PR — and then decide what it said by matching a string in its output.

`check-pr-fully-clean.py` answers twice. It prints findings for a human, and it exits 0 for clean and non-zero otherwise. Only the second is a stable interface. The prose is free to gain a line, split across two lines, or word a finding differently, and every one of those silently changes what a `grep` decides.

Two properties make this worse than an ordinary parsing slip.

It fails toward clean. The natural spelling is a positive test for the bad state — `if output matches "NOT fully clean" then not-clean, else clean` — so *any* failure of the match, including the check erroring or printing its header separately, lands in the `else` branch and reports clean. A missed match and a genuinely clean PR are the same observable, which is `fail-fast`'s pass-path-equals-failure-path shape arriving through a tool built to prevent exactly this.

It launders. The report reads as the instrument's verdict rather than as your reading of it, so "the checker says clean" is what reaches the human — and nothing in that sentence exposes that a `grep` stood between the two.

The status is three-valued, and collapsing it to a boolean is the same mistake one layer further in. `check-pr-fully-clean.py` exits **0** clean, **1** not clean, and **2** for a usage or environment error. That third code is deliberate — its own source says `USAGE_EXIT = 2` exists so "a usage or environment error would have been read as a verdict about the PR" — so `if ! checker; then not_clean` throws away the distinction the script went out of its way to provide.

The cost is a **false regression**: a transient `gh` failure, a rate limit, a network blip in a polling loop, all report a PR as having gone not-clean. That is the mirror of the grep bug above, which failed toward clean; this one fails toward alarm, and both are a two-branch reading of a three-branch answer.

This is the rule `errexit-is-not-uniform` states as 0, 1, and anything else being three answers and not two — itself a paraphrase of `fail-fast`'s hand-check guidance to treat 0 as found, 1 as clean, and anything else as the check having failed to run. It applies to a purpose-built checker exactly as it does to `grep`.

But 2 does not cover every non-verdict, so the three-way read is necessary and still not sufficient. `USAGE_EXIT = 2` is raised by `die()`, on the paths the script anticipated. An **unhandled exception** exits **1** — the code reserved for "not clean" — so a crash is indistinguishable from a verdict by status alone.

That is why the status read has to be paired with a look at the output rather than replacing it. A genuine not-clean prints `-` finding bullets; a crash prints a traceback. One `grep -q '^`

`- '` separates them, and unlike the phrase search above it is keyed on the report's *structure* rather than on its wording.

The wrong-repo case is the one to expect, because the script resolves the repo from the **current working directory** unless `-R/--repo` is passed. A background poller inherits the session's cwd, which on a multi-repo session is routinely not the repo the PR lives in — so the same command answers correctly by hand and crashes in the loop. Pass `-R OWNER/REPO` explicitly in anything that is not a one-off typed inside that checkout. See `fully-clean.cases.md`, "Checker unhandled exception on wrong repo".

A remote or web session has no `gh` at all, so the checker cannot answer there — and that is a property of the session rather than of the PR. The wrong-repo case above is a mistake you can stop making. This one is not: `check-pr-fully-clean.py` shells out to `gh`, and a remote/web Claude Code session has no `gh` on `PATH`, so the script refuses with `` `gh` is not installed or not on PATH `` and exits **2** on every invocation, whatever the PR's real state.

That lands in the third branch of the read above, which is the right answer and an easy one to skip past, because the mandated instrument failing feels like a step to work around rather than a result to report. Two things follow.

Say that the checker did not run. `ardi`'s fully-clean exit checklist opens by requiring exit `0` from it, so reporting a PR clean without noting the substitution asserts a check that never happened. The substitution itself is ordinary — root `CLAUDE.md`'s "Skills that call gh/glab: fall back to tool-mappings.md in remote sessions" already governs it — so establish both criteria from the GitHub MCP surfaces instead: the paginated check-runs endpoint for criterion 1, the review body and thread list for criterion 2.

Do not read the `2` as a verdict in either direction. It is neither "not clean" nor a licence to assume clean. It is the check declining to answer, which is exactly what the three-valued read above exists to preserve.

- **Do:** state which surfaces supplied the verdict when the checker could not run, so "clean" stays attributable.
- **Don't:** report the checklist item satisfied on a session where the script exits 2 — it did not run.
- **Don't:** treat the refusal as a PR problem, or spend a round diagnosing it; the absence of `gh` is the whole cause.

(Measured 2026-08-19 on a remote session driving ai-config#1673. Tracked as ai-config#1679, which weighs teaching the script a REST fallback against documenting the branch; until one lands, every remote-session ARDI run hits this.)

So read the status, and read all three of it:

```
python3 scripts/check-pr-fully-clean.py "$n" -R "$OWNER/$REPO" >/tmp/fc.txt 2>&1
rc=$?
case $rc in
  0) echo "#$n CLEAN" ;;
  1) if grep -q '^  - ' /tmp/fc.txt; then
       echo "#$n NOT clean"; cat /tmp/fc.txt
     else
       echo "#$n CHECK CRASHED (rc=1, no finding bullets) -- not a verdict"
       tail -3 /tmp/fc.txt
     fi ;;
  *) echo "#$n CHECK FAILED (rc=$rc) -- not a verdict"; cat /tmp/fc.txt ;;
esac
```

- **Do:** branch on the checker's exit status, treating 0 as clean, 1 as a verdict of not-clean, and anything else as the check having failed to answer.
- **Do:** re-verify the agent and the head yourself before reporting ready, since the exit status is necessary and this file's own SHA-surface caveats still apply.
- **Don't:** grep a purpose-built checker's output for a phrase — its prose is a human-facing report, not an API.
- **Do:** pass `-R OWNER/REPO` from any poller or script, since the repo comes from the working directory otherwise and a background loop inherits whatever cwd the session happened to be in.
- **Don't:** collapse the status to a boolean either; `rc != 0` reports a broken check as a regressed PR, which is the same conflation wearing the remedy's clothes.
- **Don't:** read `1` as a verdict without checking the output has finding bullets — an unhandled exception exits 1 too, so `2` is not the only non-verdict code.
- **Don't:** read "I called the right instrument" as having consumed it; the bypass guard fires on the call, and nothing fires on the misreading.

See fully-clean.cases.md, "Three PRs reported clean by grepping the checker's own output".

Exit 0 is not the whole answer either: read the `verdict scan:` line the checker prints, because it can say `0 bore a verdict, latest = NONE` on a run that exits clean. The three-way read above governs every status that is *not* 0, so it cannot reach this one — the false clean arrives as exit **0**, the one value nothing above tells you to look behind. `check_latest_verdict()` blocks on `not-clean` alone, and an empty verdict is not `not-clean`, so a head reviewed by nobody takes the clean return. A reviewer's own **skip notice** is enough to occupy the slot.

- **Do:** read the `verdict scan:` line on every invocation, including the ones that exit 0.
- **Do:** treat `latest = NONE` as no review at all, and fall back per self-review-fallback.

- **Don't:** read exit 0 as "a reviewer approved this" — it says only that nothing blocking was found, and an empty review history finds nothing.
- **Don't:** count a skip notice as the review; it is admitted as a review item and states no verdict, which is exactly the state that exits 0.

The author filter gates formal reviews and not comments, so a human-authored comment enters that same scan on body text alone. The comment loop admits on `is_bot_author or is_review_header`, and `is_review_header` matches `### verdict`, `verdict:`, and `code review` with no author check — so your own disposition comment, or any reply quoting a reviewer's verdict line, can be counted as a review item. Reading the formal-review loop and generalizing its author check to comments is `verify-the-right-artifact`'s "a neighbour for the target" shape applied to source.

- **Do:** read the loop that handles the artifact class you are making a claim about — comments and formal reviews are separate populations here.
- **Do:** check a comment's admission against its body markers, not its author.
- **Don't:** generalize one loop's filter to a neighbouring loop in the same function.
- **Don't:** read "no human comment appeared in `matching_items`" as evidence that human comments are excluded; the SHA test is what excluded it.

See `fully-clean.rationale.md` for both mechanisms, and `fully-clean.cases.md`, "A skip notice exits the checker clean over an empty verdict scan".

A verdict comment quotes verdict phrases, so a phrase search identifies nothing — and it misreads in both directions at once.

- **Do:** call `check-pr-fully-clean.py` for a sweep's verdict column, exactly as `ardi` requires for one PR.
- **Do:** anchor on the last `### Verdict` heading when parsing by hand, after selecting candidates on the `**Claude finished` marker.
- **Don't:** take the first verdict phrase in a body as that body's verdict — quoting other verdicts is part of what a review comment does.
- **Don't:** assume such a misread has a safe direction; one sweep produced a false-clean and a false-blocked.

That "anchor on the last `### Verdict` heading" line describes the by-hand method, not what `check-pr-fully-clean.py` itself does — the script has no heading anchor at all. It matches verdict *phrases* with a regex (`Verdict:\s*(?:Clean|Approved|Ready)\b` and its not-clean counterpart), never a `^###\s*Verdict` heading line, so a doubled or malformed `### Verdict` heading in a review comment cannot break something the script never checks. Reading this fragment's hand-parsing advice as a description of the script's own mechanism produces a confident, wrong claim about our own tooling — worth naming because the fragment sits right next to the script it is easy to assume it summarizes.

- **Do:** read `scripts/check-pr-fully-clean.py` itself when the claim under test is about what the script does, even when this fragment already describes the by-hand procedure.
- **Do:** treat "anchor on the last `### Verdict` heading" as guidance for a human parsing a comment, distinct from the script's own phrase-matching logic.
- **Don't:** infer the script's parsing mechanism from this fragment's by-hand advice — verify against the script's source before filing an issue that names a mechanism.

See `fully-clean.cases.md`, "A fragment's by-hand parsing advice mistaken for the script's own mechanism".

A review comment's header SHA can be stale, so take the reviewed commit from the run's own `head_sha`.

- **Do:** follow the job link in the comment and read that run's `head_sha`.
- **Don't:** treat the SHA in a comment's heading as the commit reviewed.

That remedy assumes the run checked out the PR head, and a `workflow_dispatch`-triggered review run does not.

- **Do:** check a `workflow_dispatch` review's `event` field before reaching for `head_sha` — on that trigger type the field names the dispatch ref, not the reviewed commit.
- **Do:** cross-check a stale-suspected verdict's specific claims against the file directly, rather than only against run metadata.
- **Don't:** trust `head_sha` as "the commit reviewed" on a workflow-dispatch-triggered run — that guarantee only holds for push/pull_request-triggered runs, which check out the PR head by construction.

A third surface names a commit the run never read, and unlike the two above it points the confident direction: the run object's own `pull_requests[].head.sha`.

- **Do:** settle which commit a review read from a discriminating claim in its own body, since that is the only surface separating the candidates.
- **Do:** read `pull_requests[].head.sha` as a fact about the PR's current head, useful for nothing else.
- **Don't:** read that field naming your latest commit as evidence the review covered it — it names the current head unconditionally.
- **Don't:** read an empty `pull_requests` as evidence about the run; the array empties when the PR closes.

See `fully-clean.cases.md`, "`pull_requests[].head.sha` named a commit pushed after the run started".

`check-pr-fully-clean.py` uses the same unreliable body-text surface, and whichever SHA that text happens to contain — present, absent, or wrong — is incidental to which head the run actually reviewed.

- **Do:** read a flagging run's `event`, `head_branch`, and `head_sha` before treating "no review at this HEAD" as a genuine gap.
- **Do:** treat the script's discharge as the likely reading, since the withholding direction dominates in practice, but not as a certified one.
- **Don't:** re-dispatch a review, or fall back to self-review, on this signal alone when the flagging run's own metadata already shows it evaluated the current head.
- **Don't:** read a body's SHA, present or absent, as evidence about which head a review covered — it is evidence about what the prose happened to discuss.
- **Don't:** conclude that reviewers citing their head SHA more consistently would fix this; a body can already cite a SHA and still be citing the wrong one.

A clean CI run and a clean review verdict are a snapshot, not a standing guarantee of mergeability. `main` can advance after your last check — including gaining its own independent addition that collides with yours (see `sync-with-main.md`'s "two PRs append the same numbered subsection" case) — so re-verify the branch still merges cleanly against current `main` before reporting a PR ready, not just trust the last green run.

Re-check version parity in that same sweep, not only conflict-freedom.

Threads: at fully-clean, every **inline** review thread is resolved, and the only conversation left open is the final all-clear exchange — the reviewer's all-clear comment and your reply to it. (The all-clear is usually a top-level PR comment, not an inline thread.)

One finding can own two threads, so sweep by thread id rather than by finding.

Deadlock -> escalate to a human. If you and the reviewer(s) can't reach consensus on an item (a rebuttal was exchanged and neither side is budging), don't loop forever and don't unilaterally override the reviewer — request a **human reviewer**, `@`-mention them in a comment summarizing the impasse, and surface the open item.

An automated reviewer's verdict on a disputed factual/technical claim is not stable across independent runs, even with identical evidence available each time. Don't treat one round's "settled, no need to keep arguing" as durable: the very same review job, re-triggered later with no new code changes, can re-raise a claim it previously retracted — and then retract it again on a subsequent run — purely from re-deriving the question differently each time, not from anything changing in the PR. This means a rebuttal thread's outcome (however many rounds of citations and counter-citations) doesn't itself resolve a genuine deadlock the way a human's decision does; only escalating per the bullet above actually settles it. The one thing that DOES help going forward: fold the authoritative citation/evidence directly into the code or doc being reviewed (a comment, not just a PR conversation reply) — a fresh reviewer run re-deriving the claim from scratch is more likely to find the citation sitting right next to what it's evaluating than to dig through prior thread history for it, though even that is not a guarantee against a bot that ignores context already in front of it.

Address Every In-Scope Review Comment

When iterating on a PR with a reviewer, **address every in-scope flagged item**, regardless of severity label. The reviewer's "Not a blocker", "minor", "nit", "optional", "consider", or "if you want" labels are for prioritization, not a free pass for the implementer.

Extended rationale — the mechanism, evidence, and argument behind each rule below — lives in `address-every-comment.rationale.md`, moved out of the auto-loaded context. Each rule here keeps its statement and its Do/Don't pair; read the companion when the reasoning or the evidence is the question.

Worked-example case records for the rules below live in `address-every-comment.cases.md`, moved out of the auto-loaded context.

For each flagged item, do exactly one of:

1. **Fix it in this PR.** The default path — most nits are 1–3 line changes.
2. **Defer.** Only when the fix expands the PR's scope (new feature, broader refactor, separate concern), the requester has explicitly said this PR shouldn't grow, or the flagged content isn't actually yours to fix here (see the `main`-sync case below). File a follow-up issue and reference it in a PR comment so the item isn't lost — except in the `main`-sync case, where the "follow-up" is fixing it on `main` directly, not a new issue.

Then trigger another review and repeat until the PR is **fully clean** — zero flagged items under any heading, no "non-blocking", "harmless", "minor observation", or "could improve" sections. "Looks good" / "no findings" / "approved" with no follow-on bullets is the bar. Resolve every inline review thread along the way, leaving only the final all-clear exchange.

Always resolve an inline thread the moment its comment is successfully addressed — the fix pushed and a reply posted naming it — in the same pass, whatever workflow you're in: a formal `ard`/`ardi` round, a CI-monitor nudge, or a one-off fix outside any loop. Addressing without resolving leaves a thread that reads as outstanding work to every later reviewer, blocks `fully-clean`'s every-inline-thread-resolved criterion, and drags stale noise into the next review round. The per-disposition settlement rules in `ard` step 4b still govern the exceptions: a **Rebut** stays open until the reviewer drops it, and an **Address** you're not confident fully settles the concern gets a reply asking for confirmation instead of a resolve. The `resolve-pr-threads` skill sweeps any stragglers, but it's a backstop — resolve-on-address is the default, not a cleanup step.

Do **not** report "ready to merge with one minor nit noted" / "harmless as-is" / "can address if you want" — that hedging just pushes triage back to the requester.

A round count is never a reason to stop, and "the reviewer keeps finding things" is not a finding about the reviewer. There is no threshold after which unaddressed items become acceptable: keep requesting reviews and keep dispositioning findings until a review comes back with none. The only exits are a totally clean review, a genuine per-item deadlock,

or the user calling it. Reasoning of the form "we have done N rounds, shall we accept the current state?" is the same hedging this paragraph bans, moved up from one finding to the whole loop — see `ardi`'s "Stopping conditions" for why it fails and for the case record.

Noise is per-item, not per-round — don't stop the whole loop over one recurring flag. A long-running PR can have both real findings (worth fixing every round) and one specific item the reviewer re-raises verbatim round after round even though it's already deferred/tracked (e.g. a file-length guideline already split into a follow-up issue). Keep fixing every *new* finding as it appears — don't let the recurring item make you stop processing genuinely new ones. But stop re-litigating *that one item* every round: reply once pointing at the tracked issue, and hold on it specifically rather than re-deferring it on each pass. Surface the pattern to the user (which item, how many rounds, where it's tracked) and let them decide whether to resolve it now (e.g. do the split) or leave it as accepted recurring noise — don't decide unilaterally to either keep re-processing it or silently drop it.

When a finding is a pattern (a formatting/style rule broken in one spot), apply it everywhere it recurs in the same file, not just the flagged line.

That rule's scope is "the same file", and a reviewer who enumerates the sites is the reason the scope goes unquestioned.

- **Do:** derive the site list by grepping the whole diff for the flagged phrase, and fix what the grep returns.
- **Do:** report the sweep — the pattern searched and the hit count — rather than the number of sites you fixed.
- **Don't:** treat a reviewer's enumeration as the extent of the pattern; it is the extent of that reviewer's read.
- **Don't:** write "all N spots you named" into a reply, since quoting the reviewer's count is the tell that no sweep ran.
- **Don't:** read a null result as "no further sites"; it means no further hit for that pattern, and a differently-worded instance would not have matched.

The remedy above names a search space — "the whole diff" — and a stack has one of those per branch.

- **Do:** run the derived sweep over every branch in the stack, and report the per-branch counts.
- **Do:** treat a finding that names a convention as scoped to the work rather than to the PR it was filed on.
- **Don't:** read "the whole diff" as satisfied by the diff of the PR the reviewer commented on — that is one of N.

See `address-every-comment.cases.md`, "A finding's site list spans every branch in the stack".

Deriving the class is necessary and not sufficient, because you can derive the wrong one — and the growth rate across rounds is what says so.

- **Do:** read growth in the list across rounds as evidence about the lever, not as a count of members still to add.
- **Do:** state a reviewer's diagnosis back in your own words before acting on its examples, so a redirect cannot be worked past in silence.
- **Do:** relax the enumeration and read which cases move — a property shared among them names the axis.
- **Don't:** treat "I derived the class rather than fixing the reported instances" as discharging the rule above; it is that claim one level up, and it fails the same way.
- **Don't:** prefer the actionable half of a review to the diagnostic half merely because it is the half you can start on.

A narrower version of the same failure: the class is right, and it is enumerated in more than one place.

- **Do:** paraphrase the last two or three findings into a single sentence, and read a match as evidence that a concept is duplicated rather than incomplete.
- **Do:** derive how many sites encode the concept, then consolidate them into one definition every site consumes.
- **Don't:** answer a third instance by extending a third list — that is the same round again with a new door.
- **Don't:** skip the review's own prose naming a sibling site; it is frequently there, in the paragraph explaining why some other mechanism did not save you.

The mirror case: the enumeration was complete and the fix was not.

- **Do:** count the artifacts a single comment names, and give each one its own disposition before replying.
- **Do:** read the rendered page rather than the diff when confirming that a prose or formula fix landed completely.
- **Do:** grep the whole file for the underlying concept once a second half surfaces — a document stale in two places is usually stale in three.
- **Don't:** let a visibly-changed flagged line stand in for the finding being closed; the unfixed half appears in the diff as context.
- **Don't:** reach for the derive-the-site-list remedy above here — that list was complete, and the shortfall was in the delivery.

When a prose fix changes wording that's also paraphrased elsewhere in the same PR (a CHANGELOG entry, a PR description, a cross-reference), sync that copy too. A CHANGELOG entry written before the review lands often quotes or paraphrases the exact phrase a reviewer later flags; fixing the source prose but leaving the paraphrase stale

reintroduces the same wording issue one file over. Grep the diff for the flagged phrase before considering the finding closed.

A scope-widening fix makes its stale copies invisible to every diff-scoped sweep, so there the search space is the whole file, not the diff. The whole-diff rules below are right for a fix that changes a claim's wording: the synced copies entered the diff when you edited them. A fix that *broadens* a concept's scope inverts that. The restatements that are now too narrow are precisely the lines the fix did **not** touch, so they appear in the diff as context or not at all, and an added-lines sweep structurally cannot see them — it reports a confident zero over exactly the population the finding is about. Grep the whole file (and any file restating the concept) for the concept that widened, and read each hit against the new scope.

- **Do:** after a broadening fix, sweep every restatement of the concept in the whole file, not the diff's added lines.
- **Don't:** read a clean added-lines sweep as closing a broadened-scope finding — the stale copies are unchanged lines by construction.

(Morrison-Lab/ai-config#1490, 2026-08-15/16: rounds 1, 2, and 4 each found a sentence still Copilot-only after the surrounding passage was broadened to cover human reviews. The round-2 fix swept the diff's added lines for `Copilot` — 17 hits, all legitimately Copilot-specific — and round 4 still found an un-broadened copy in the `## Output` section, because that copy was an unchanged line the added-lines sweep could never have matched.)

When syncing copies, search the diff for the claim, not the files or symptom already in front of you.

- **Do:** run whole-diff searches for synchronized figures and phrases, after committing the fix, and report the before/after counts.
- **Do:** when a rationale is retired, search for every wording that states that rationale or criterion, not only for the symptom word that made it fail.
- **Don't:** substitute `grep -rn <term> <files-you-had-open>` for grepping the diff.
- **Don't:** accept a search for the visible contradiction as proof that the retired claim itself is gone.

That same search settles a narrower question about placement: a correction written NEAR the flagged sentence reads as having replaced it, while the flagged sentence survives and the file then states both. Edit the sentence the reviewer named, and use the search above — for the claim, not for the symptom — to confirm that wording is gone.

- **Do:** delete or rewrite the flagged sentence itself, rather than adding a truer one beside it.
- **Do:** mark superseded text as superseded, explicitly, where it is worth keeping as a record of why something was done.
- **Don't:** read "the file now contains a true sentence" as having addressed a finding about a false one.

- **Don't:** let a commit message assert a deletion that the diff shows as an addition beside unchanged text.

See `address-every-comment.cases.md`, "A correction added beside the flagged sentence, which survived".

When the wrong thing is a figure, the unit of repair is the figure — across every artifact carrying the twin, not just the diff.

And a reflow puts its neighbouring sentences into your change, for fact-checking and not only for lint.

- **Do:** grep for the figure's value across every artifact carrying the twin, before replying that the finding is closed.
- **Do:** fact-check the sentences a reflow pulled into your diff, exactly as you would the ones you wrote.
- **Don't:** treat the named occurrence as the unit of repair when the same value appears elsewhere.
- **Don't:** read a clean whole-diff grep as covering a twin the diff never touched.

See `address-every-comment.cases.md`, "The unit of repair is the figure, across every artifact carrying the twin".

The PR description is on that list and is the one copy grepping the diff cannot find, so check it separately.

- **Do:** re-read the PR description after any Address that changes what the PR does or why, alongside the changelog check above.
- **Don't:** treat a clean `grep` over the diff as evidence every paraphrase is synced — the description was never in it.

Answering a body-staleness finding with a correction comment does not clear it, and this corpus's own visible-correction convention is what makes that move attractive.

- **Do:** edit the body **and** record the correction inside it, so nothing is silently overwritten and earlier rounds still resolve.
- **Don't:** answer a body-staleness finding with a comment — the next reviewer re-reads the body, so the finding survives it.
- **Don't:** treat the drift risk in rewriting a long body as a reason to leave it; re-deriving every figure is what the round already requires.

See `address-every-comment.cases.md`, "A body-staleness finding is answered by editing the body".

A body that reports volatile external state goes stale with no edit of yours, so the trigger above never fires on it.

- **Do:** describe the change, and let CI report CI.
- **Do:** timestamp and scope any status you must state ("red as of `<sha>`, cause was X"), so it cannot be read as a present claim.
- **Don't:** put current CI status, mergeability, or a blocker in a PR body undated — the body is the one place nothing re-measures.
- **Don't:** expect the "after any Address" trigger to catch it; that fires on your edits, and this goes stale without one.

Following that "state it as history" advice is what produces the next block, because an automated reviewer reads the body as a flat statement of intent.

- **Do:** state the current content first, marked as current, before any history.
- **Do:** put the reversal in its own section that opens by saying it is history.
- **Do:** make sure the "what is excluded" section does not name the reversed item at all, in any tense.
- **Don't:** rely on past tense alone to carry the distinction.
- **Don't:** revert a maintainer-requested change because a reviewer read the history as current — rebut, and escalate rather than comply.

The same sync is needed when the review fix is to CODE BEHAVIOR rather than to wording — and that case is easier to miss, because nothing about fixing a bug points at the changelog.

Tighter still: a changelog entry can contradict its own commit message, in the same commit, with no review in the loop at all.

One step further back: a figure inherited from the tracking issue is both the copy git keeps and the copy nobody verified.

- **Do:** re-run the check when a figure moves from an issue into a commit message, even having verified it once for the PR body.
- **Do:** read `git log -1 --format=%B` before pushing, against the same source the body's claims came from — a commit message is not greppable from the working tree once written.
- **Don't:** copy a count, version, or path out of the tracking issue on the strength of having written that issue.
- **Don't:** treat "permanent in history" as settled while the PR is unmerged — `git commit --amend` still works, and is usually worth a fresh CI round against a wrong figure reaching `main`.

A corollary for checking any of this in a semantic-line-break corpus: a single-line `grep` returns false negatives on your own prose.

Inline markup breaks the same search, and that variant aims the false negative at someone else's work rather than your own.

- **Do:** account for inline markup as well as whitespace before concluding a quoted phrase is absent — see the next block for which side to normalize.
- **Do:** read the single hit when a search for a citation's target returns only the citation itself.
- **Don't:** file a dangling-citation issue while the only evidence is a literal grep that found nothing but the citation — that is the search failing, until a normalized one agrees.

Apply whatever normalization you choose to the search term as well as to the text, or the fix produces a third false negative of its own.

- **Do:** normalize the needle with the identical function applied to the text, so the comparison is between two transformed strings.
- **Do:** re-test any earlier absent verdict after extending a normalizer, since the extension can break a term the previous version matched.
- **Don't:** enumerate which markup to strip and treat that list as the fix.
- **Don't:** test a raw search term against normalized text, however plain the term looks.

Symmetry is necessary and not sufficient once the haystack is source code, because a line-comment leader is inserted by the medium rather than by the author. The rule above governs inline markup — backticks, asterisks, underscores — which an author types *inside* a phrase, so stripping it with a character class is the right shape. A `##`, `#`, `//`, or `--` leader differs in two ways that each defeat that class. It appears at a **line start** rather than mid-token, so it interrupts a phrase only where the phrase happens to wrap. And `#` is not in the class at all, so applying the same normalizer to both sides leaves it in the haystack and absent from the needle — which is exactly the asymmetry the rule was written to remove, arriving through a character nobody enumerated.

The failure direction is the expensive one. A verbatim phrase that *is* present reports absent, so the natural response is to re-add content that was never missing.

Widening the class is the wrong repair, and the Do/Don't block one paragraph up already says so: **don't** enumerate which markup to strip and treat that list as the fix. The rationale companion puts it more sharply — "Enumerating is the wrong shape, not merely an incomplete list." Adding `#` to `[\`*_\s]` also strips a `#` a phrase legitimately contains — an issue reference, a colour literal, a quoted shell comment — so the normalizer starts erasing content in order to find it.

Strip the leader **per line, anchored**, before collapsing whitespace:

```
strip_leader = lambda s: re.sub(r"(?m)^[ \t]*(##|#|//|--)(?=[ \t]|$)[ \t]?", "", s)
norm = lambda s: re.sub(r"[\`*_\s]+", " ", strip_leader(s))
norm(needle) in norm(haystack)
```

The anchor is what keeps this from being the wider-class move. `^` under the `(?m)` flag confines the strip to a position the medium owns, so a `#` inside a line is untouched.

The lookahead is load-bearing rather than decorative, and dropping it reintroduces the exact defect this section removes. A bare optional separator (`\s?`) lets the pattern strip any line-initial `#` or `--` whatever follows it: the `#` of a wrapped `#1257` reference, and — worse in a corpus that writes them constantly — a line-initial `---`, which is left as a stray `-`. Requiring the separator to be present, or the line to end there, leaves both intact while still stripping `## text`, `-- text`, and a bare `##`. Prefer `[ \t]` over `\s` for that separator, since `\s` matches the newline and would join the stripped line to the next one.

- **Do:** strip a line-comment leader with an anchored per-line pattern before whitespace collapse, whenever the haystack is source code.
- **Do:** apply that strip to both sides — this adds a stage, it does not replace the symmetry rule above.
- **Do:** require the leader to be followed by whitespace or a line end, so a line-initial `#1257` or `---` survives the strip.
- **Don't:** add `#`, `/`, or `-` to the inline-markup character class; that strips them wherever they appear, including inside the content you are searching for.
- **Don't:** read an absent verdict against a source file as evidence the phrase is missing until the leader has been accounted for.

(2026-08-16, verifying `Lacaedemon/sparta` PR #1257 after merge: a probe checking that two merged doc-comment phrases had landed on `main` reported both missing. Both were present. Each phrase wraps across lines in `scripts/SoldierEnemyContact.gd`, and every continuation line opens with GDScript's `##` doc-comment leader, so the haystack carried `##` mid-phrase where the needle carried a space. The normalizer was applied to both sides, exactly as the rule above requires, and `#` is not in its character class — so the symmetry held and the check still failed.)

A flagged item that came in via a `main`-sync merge, not your own diff, is still a Defer — just one where the follow-up is fixing it on `main` directly, not filing a per-PR issue. This is not the ARD skill's "Acknowledge" disposition: `skills/ard/SKILL.md` reserves Acknowledge for praise or a no-ask observation, and explicitly warns against stretching it to dodge a real finding — a redundant config line a reviewer flags is a real finding with an implied fix request, so it needs a real disposition, not a label that means "no change requested." When a reviewer flags something (a redundant config line, a stale pattern) inside a file your branch only touches because you merged `main` in to resolve a conflict, check provenance before fixing it: `git log`/`git blame` the flagged line, or just compare against `origin/main`'s current content. If it's identical to `main`, "fixing" it on your branch alone doesn't fix anything — it just makes your branch disagree with `main` on unrelated content the next person to touch that file will have to reconcile again. Reply agreeing the finding is correct but out of scope for this PR, and leave it for whoever owns that file's actual content to fix on `main` directly — no follow-up issue needed, since the fix target is `main` itself, not this PR's own change.

This generalizes to a skill's own inline restatement of a fragment it links to. A `SKILL.md` that links a backing `shared/` fragment for the full detail often *also* restates the

fragment's approach or word list inline (in its `description` field, or a short procedure-step summary) so a reader doesn't have to open the linked file. Fixing a bug in the fragment doesn't automatically fix these inline restatements — they're a second, independent copy of the same claim, and a review round after the fragment fix can catch them going stale exactly like a CHANGELOG paraphrase does. Grep the whole PR diff for the fixed phrase/word-list, not just the fragment file, before considering a fragment fix complete.

A bot that re-raises an item as "not addressed" may simply not have seen your reply — check the timestamps before treating it as an impasse. An automated reviewer gathers the PR's comments once, when its run starts. A rebuttal posted after that snapshot is invisible to it, so the next round reports the item as still open and unaddressed even though a substantive reply is sitting in the thread. The tell is a re-raise that repeats the original finding verbatim and speaks only to whether the *code* changed, without engaging any argument you made. Before escalating, compare your reply's timestamp against the review run's `started_at` (`gh run view <id> --json startedAt`, or the `started_at` field each run carries in `get_check_runs` when `gh` is absent): if the reply landed after the run began, it is a stale re-raise, not a genuine disagreement.

Reply-first collides with citing the fix's SHA, and the way out is to commit between them rather than to pick one.

1. **Commit** the round's fixes. The SHA now exists and is stable.
2. **Reply** on each thread, citing that SHA.
3. **Push.** The next review's snapshot already contains the replies.

- **Do:** commit, reply citing the committed SHA, then push — in that order.
- **Don't:** treat "I need the SHA for the reply" as a reason to push before replying; that is the ordering the bullet above exists to prevent.

A finding can be right while its `suggestion` block is wrong — verify the suggested literal before applying it.

The same check applies to a fix a reviewer describes in prose rather than in a `suggestion` block, and the sharpest test is the reviewer's own example.

A reviewer's corrected citation is another factual claim, so verify the replacement before adopting it.

- **Do:** verify a proposed replacement citation with the source's own history before editing the PR to use it.
- **Do:** use `git log -S "<exact line>" -- <file>` or an equivalent provenance query when the question is which PR introduced text.
- **Don't:** adopt a reviewer's corrected issue or PR number because the original was wrong.
- **Don't:** use word overlap and same-day timing as a substitute for source history.

The same check one artifact over: a reviewer's replacement DIFFSTAT is a factual claim too, and the usual way it goes wrong is summing per-commit churn rather than diffing the merge base.

A branch that edits the same lines across review rounds accumulates churn. Round 1 adds a line and round 2 rewrites it, so a per-commit sum counts that line twice and reports a deletion the merge-base diff never sees. The inflation is therefore worst on exactly the branches most likely to carry a verification table worth checking, which are the multi-round ones.

What makes this cost more than one wrong number is that `ardi`'s pre-push checklist already requires every figure in a PR body to be re-derived by command at each push. A reviewer supplying replacement figures looks like that derivation having been done for you, so the natural move is to paste them straight in. That substitutes an unverified figure for a stale one and leaves the body just as wrong, while feeling like the finding was addressed.

- **Do:** re-derive a reviewer's replacement figures with `git diff --numstat <merge-base> <head>` before pasting them into a PR body.
- **Do:** cross-check against GitHub's own `additions`/`deletions` fields, which are computed against the merge base and so agree with that command.
- **Don't:** treat a reviewer's supplied figures as discharging the re-derive requirement — a correct finding about staleness says nothing about the replacement's accuracy.
- **Don't:** sum per-commit `--numstat` to get a branch's diffstat. On a multi-round branch that double-counts rewritten lines and reports deletions the merge base never sees.

See `address-every-comment.cases.md`, "A reviewer's replacement diffstat summed per-commit churn".

The highest-yield version of that check: when a comment names an edge case in its own prose and also supplies a fix, run the fix against that edge case.

- **Do:** check a suggested fix against every failure mode the same comment names, before checking anything else about it.
- **Do:** name the reviewer's own caveat in the reply, so the rebuttal rests on their evidence rather than on your say-so.
- **Don't:** let a comment's demonstrated thoroughness transfer to its snippet — they are separate claims.
- **Don't:** discard a finding because its fix is wrong; the half that named the hazard usually still stands.

A quieter variant: the suggestion introduces no defect at all, it restates the line above it — so applying it deletes coverage while reading as hardening.

- **Do:** evaluate the suggested predicate and its neighbours on real input, and keep the finding while rejecting the snippet when they coincide.
- **Do:** fix the underlying coupling instead, and say in the reply why the suggested form was set aside.

- **Don't:** accept a `suggestion` block that restates an adjacent check — passing tests afterward prove nothing, since the survivor passes for both.
- **Don't:** read a reviewer's own "the line above already covers this" as support for their replacement.

A finding can be right, and its fix adequate, while the *reason* it supplies is too weak to ship — and in a corpus of rules, the reason is the deliverable.

- **Do:** read the primary source for the strongest reason before adopting a suggested rationale, even when the suggestion's conclusion is right.
- **Do:** say in the reply which reason you took and why the offered one was set aside, since deviating from a `suggestion` block silently reads as having missed it.
- **Don't:** accept a defensible-sounding mechanism because the conclusion it supports is correct.
- **Don't:** treat this as grounds to reject the finding — the conclusion usually stands, and only its reason needs strengthening.

And the mirror case: a finding can be wrong on its stated grounds while still pointing at something real.

A third direction, which evades the verification reflex rather than lacking a rule: agreeing with a finding and then escalating it.

- **Do:** verify an escalation against the full scope it claims, which is wider than the scope the finding reported, and which the finding's own instrument may already cover.
- **Do:** post the correction to the thread that carried the escalation.
- **Don't:** treat agreeing-and-extending as exempt from the checks a rebuttal gets, since agreement suppresses the reflex that disagreement triggers.
- **Don't:** report a finding as understated on a measurement you have not shown covers the whole field set.

When a finding cites a source, read the cited source before reproducing anything – it is the cheaper instrument, and it is the one that can show the finding backwards rather than merely unsupported.

When a reviewer hedges a finding because it depends on code it cannot see, check whether *you* can see it — the hedge is an invitation, not a verdict.

Timestamp the evidence before rebutting a finding with it — during a live incident, a log from twenty minutes ago describes a different system.

A rebuttal's own evidence is the least-checked claim in a review round, and the commonest way it goes wrong is being measured through a tool that adds a shell layer.

- **Do:** write each command spelling to its own file and run the file when comparing them, so exactly one shell layer applies.
- **Do:** hold your own rebuttal to the standard you would apply to the finding, and say which instrument produced the counter-measurement.
- **Do:** re-run the measurement outside the harness when a reviewer holds their ground, before rebutting a second time.
- **Don't:** read a rebuttal as self-verifying because disagreeing felt like the rigorous move.
- **Don't:** cite a named check as settling a question without saying what it ran through; a named check reads as a performed one.
- **Don't:** compare two command spellings by typing both into the same tool, which is the one measurement guaranteed to make them look alike.

A finding carries a timestamp too, and its precondition can dissolve between the round that raised it and the round that addresses it.

The claim is not thereby fixed either, which is the half that is easy to miss.

- **Do:** re-check what a finding presupposes at the moment you address it, not at the moment it was raised.
- **Do:** replace a claim whose gate has cleared with a derived one — the timestamp and the figures the event produced — rather than hedging it or leaving it.
- **Don't:** apply a reviewer's suggested wording without re-checking what that wording presupposes; a hedge is false once the thing it hedges has happened.
- **Don't:** read a claim that has become true as a claim that has been checked — nothing verified it, and the two read identically.

Four neighbours sit close enough to be mistaken for this, and the boundary is worth drawing because three of them fire on the same PR.

See `address-every-comment.cases.md`, "A finding's precondition can dissolve before you address it".

A finding built on a *negative* result – "I searched and it isn't there" – is only as strong as the paths that were searched, and the search scope is the part reviewers state loosest.

- **Do:** ask which paths a negative finding actually searched, and check the obvious location yourself before editing anything.
- **Do:** name the gap when the thing does exist – paths searched versus where it lives – so the same search is not re-run the same way.
- **Don't:** accept "it isn't there anywhere" as settled because it is stated more confidently than a positive finding would be.
- **Don't:** discard the finding once its negative result is disproved – the thing it tripped over is often a real ambiguity.

A note the reviewer declined to raise is still a claim, and so is your refutation of it.

- **Do:** verify a declined, out-of-scope, or passing note against the code before either acting on it or writing it off.
- **Do:** hold the change regardless when the note turns out correct but genuinely optional — verifying decides what is true, not what ships.
- **Don't:** treat a PR title, commit subject, or changelog line as evidence about what the code does; each states an intent, and a refactor can keep the very thing it says it replaced.
- **Don't:** let your own refutation past the check you would have applied to the reviewer's finding — it is a fresh claim, and overturning something feels like having verified it.

Count a round's findings before pushing its fix, because disposing of one correctly generates no evidence about the others.

The rule above governs the finding you decline to act on. This governs the finding you never see, having already acted on its sibling.

A round can carry several findings, and acting on one produces every artifact that handling the whole round produces: a verified claim, a commit, a reply, a resolved thread. Completeness is a property of the **set**, so nothing in that sequence reports that a second finding existed. There is no moment that feels like stopping early, because each step was performed properly — which is why this needs a count rather than more care.

The body-only finding is where it hides, and `fully-clean` already names why: a finding about something the diff did not touch cannot be attached as an inline comment, so it appears in the verdict body alone. Inline threads produce a visible checklist and a body-only finding produces nothing to tick off, so "all threads resolved" reads as "round handled". A **PR title** is the pure case, being out-of-diff by construction — and on a multi-commit PR a squash merge takes its commit subject from that title under GitHub's default, so an overclaiming title can outlive the PR page it was raised on.

The remedy is mechanical, and it is a count rather than a judgment: before pushing, re-read the verdict body **and** re-fetch the thread list, then state how many findings the round raised and dispose of all of them in one push. Say explicitly which are deferred, per `issue-first`.

- **Do:** state the round's finding count before pushing, derived from both the body and the thread list.
- **Do:** read a title, a changelog line, and a PR body as reviewable surfaces — a finding about any of them can only arrive in the body.
- **Don't:** read "every thread is resolved" as "every finding is handled"; the thread list cannot see an out-of-diff finding.
- **Don't:** treat a correct, complete disposition of one finding as evidence about the round — that is a per-finding claim wearing a per-round shape.

(Measured twice within half an hour on 2026-08-21, in both available shapes. On ai-config#1833 round 1 posted two inline findings; the first was fixed and pushed, and round 2 opened by re-raising the second — "the text at this location is essentially unchanged from what was flagged before" — at a cost of $2.20. On gha#550 round 1 posted three inline findings and a fourth in the verdict body only, about the PR title claiming work that had been deferred to another issue. All three threads were addressed, resolved, and pushed; the body-only one was missed. The second occurrence came after the first had already been written up, which is the argument for a count rather than for intending to look harder. The two are anchored by the re-raise at 17:12:39Z and by noticing the second miss at 17:41:36Z — derived from the PR timestamps rather than carried over from a figure quoted in a live comment, which is how "ninety minutes" reached the first draft of this entry.)

Reviewing AI-generated work

A reviewer of AI-generated work is not there to confirm that it sounds right. The job is to **try to invalidate it**. Assume the output is wrong until a check shows otherwise, and search mercilessly for mistakes.

Plausible, fluent prose is the main risk, not a comfort. It conceals errors that a human's awkward draft would have made obvious. Put on the harshest critic hat, especially for:

- citations, DOIs, and URLs that may have been invented
- functions, flags, and APIs that may not exist on the version you actually run
- numbers, file paths, and "as of" claims that were never measured

Author-side validation (Responsibility for validation) is necessary and not sufficient. A second person (or a later pass by the author wearing a reviewer hat) should still try to break the work.

Automated review is a filter, not a substitute for this stance. Those tools miss domain errors and sometimes invent findings; they do not license a lighter human review.

Auto-fix PRs with Claude Code

Claude Code's `/autofix-pr` watches the current branch's pull request from Claude Code on the web and pushes fixes when CI fails or reviewers leave comments. It detects the open PR from your checked-out branch via `gh pr view`; to watch a different PR, check out its branch first. By default it fixes every CI failure and review comment; pass a prompt to scope it, for example `/autofix-pr only fix lint and type errors`. It requires the `gh` CLI and access to Claude Code on the web. A Marketplace action at pr-autofix-with-claude-code offers the same capability as a GitHub Action.

References